

CSE 219 — Signals and Linear Systems

Coding Problem Set: Signals and Their Properties

A set of ten programming problems (with reference solutions)

Built on *Lecture 1 — A Little Introduction to Signals*

About this set. These ten problems are designed around the ideas developed in Lecture 1: continuous- and discrete-time signals, *energy* and *power*, periodicity, the transformation toolkit (time shift, time reversal, time scaling, amplitude scaling and the general affine map $x(\alpha t + \beta)$), and the even/odd structure of signals. Periodic sinusoids and exponential signals appear throughout. Each problem is self-contained and is meant to take a well-prepared student roughly **25–30 minutes**. Every problem ships with a Python template to fill in; the reference solution, the expected console output and a representative figure follow immediately after each problem.

Environment. Python 3 with `numpy` and `matplotlib` (`scipy` is not required). Implement only the functions marked `ANSWER IMPLEMENTATION`; the base signals and most plotting helpers are already provided. Numerical integration should use `numpy.trapezoid` (older NumPy: `numpy.trapz`); resampling of a sampled continuous-time signal should use linear interpolation via `numpy.interp`.

#	Topic	Core lecture idea
1	Energy, power & signal classification	energy/power definitions
2	General affine transform $x(\alpha t + \beta)$	shift–reverse–scale recipe
3	Discrete-time periodicity & period	rational $\Omega/2\pi$
4	Energy of infinite-support signals	finite energy, infinite range
5	Echo / multipath signal & its energy	time shift (delay)
6	The time-scaling energy law	time compression/stretch
7	Energy split into even/odd parts	even/odd decomposition
8	Power of periodic signals	one period is enough
9	Discrete vs. continuous energy	sampling / Riemann sum
10	Building a signal from a prototype	amplitude + shift + reversal

The four transformation problems (2, 5, 6, 10) reuse a common toolkit (interpolate, shift, reverse, scale) so the skill compounds across the set.

Problem 1: Energy, Power, and Signal Classification (Marks: 10 Suggested time: 28 minutes)

Every signal in Lecture 1 was measured by “how large” it is: we square the values and add them up (energy), or average that over the interval (power). Recall

$$E = \int_{t_1}^{t_2} |x(t)|^2 dt, \quad P = \frac{1}{t_2 - t_1} \int_{t_1}^{t_2} |x(t)|^2 dt,$$

with the sum/mean versions in discrete time. A signal is called an *energy signal* if its total energy is finite (then its power is 0), a *power signal* if its energy is infinite but its average power is a finite positive number, and *neither* otherwise.

Tasks.

1. Implement `energy_ct`, `power_ct` (trapezoidal integration) and `energy_dt`, `power_dt`.
2. Implement `classify(energies, powers)` which receives the energy and power measured over a sequence of *growing* windows and returns "energy", "power", or "neither" by looking at whether the energy has *settled* and whether the power has settled to a positive value.
3. Run the classifier on the four continuous-time signals ($e^{-|t|}$, $\cos(\pi t)$, the constant 1, the ramp t) and the three discrete-time signals ($(0.8)^n u[n]$, $\cos(\pi n/6)$, the ramp n).
4. Produce a figure showing energy-vs-window and power-vs-window for the continuous-time signals, and (in a comment) justify each verdict analytically.

Marks: energy/power functions 3, `classify` 3, all seven verdicts correct 2, plots 2.

Provided template (fill in the functions marked ANSWER IMPLEMENTATION)

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # =====
5 # ANSWER IMPLEMENTATION (fill in the four functions below)
6 # =====
7
8 def energy_ct(t: np.ndarray, x: np.ndarray) → float:
9     """Energy of a sampled CT signal: integral of |x|^2 dt (use np.trapezoid)."""
10    raise NotImplementedError
11
12
13 def power_ct(t: np.ndarray, x: np.ndarray) → float:
14     """Average power of a sampled CT signal: energy / (length of interval)."""
15    raise NotImplementedError
16
17
18 def energy_dt(x: np.ndarray) → float:
19     """Energy of a DT signal: sum of |x[n]|^2."""
20    raise NotImplementedError
21
22
23 def power_dt(x: np.ndarray) → float:
24     """Average power of a DT signal: mean of |x[n]|^2."""
25    raise NotImplementedError
26
27
28 def classify(energies: np.ndarray, powers: np.ndarray) → str:
29     """Given energy/power measured over a sequence of GROWING windows, return
30     one of "energy", "power", or "neither".
31         energy signal : energy converges to a finite limit, power → 0
32         power signal : energy grows without bound, power → finite > 0
33     """
34    raise NotImplementedError

```

```

35
36
37 # -----
38 # Test signals (given)
39 # -----
40 def make_ct_signals():
41     return {
42         "e^{-|t|}": lambda t: np.exp(-np.abs(t)),
43         "cos(2*pi*0.5*t)": lambda t: np.cos(2 * np.pi * 0.5 * t),
44         "constant 1": lambda t: np.ones_like(t),
45         "t (ramp)": lambda t: t.astype(float),
46     }
47
48
49 def make_dt_signals():
50     return {
51         "(0.8)^n u[n]": lambda n: np.where(n >= 0, 0.8 ** np.abs(n), 0.0),
52         "cos(pi n / 6)": lambda n: np.cos(np.pi * n / 6),
53         "n (ramp)": lambda n: n.astype(float),
54     }
55
56
57 def analyse_ct(xfunc, half_widths):
58     Es, Ps = [], []
59     for T in half_widths:
60         t = np.linspace(-T, T, int(2000 * T) + 1)
61         x = xfunc(t)
62         Es.append(energy_ct(t, x)); Ps.append(power_ct(t, x))
63     return np.array(Es), np.array(Ps)
64
65
66 def analyse_dt(xfunc, half_lengths):
67     Es, Ps = [], []
68     for M in half_lengths:
69         n = np.arange(-M, M + 1)
70         x = xfunc(n)
71         Es.append(energy_dt(x)); Ps.append(power_dt(x))
72     return np.array(Es), np.array(Ps)
73
74
75 def main():
76     ct_windows = [5, 10, 20, 40, 80]
77     dt_windows = [24, 120, 600, 1200, 3000]
78
79     print("=== Continuous-time signals ===")
80     for name, f in make_ct_signals().items():
81         E, P = analyse_ct(f, ct_windows)
82         print(f"{name:22s} | E={E[-1]:.4g} P={P[-1]:.4g} → {classify(E, P)}")
83
84     print("\n=== Discrete-time signals ===")
85     for name, f in make_dt_signals().items():
86         E, P = analyse_dt(f, dt_windows)
87         print(f"{name:22s} | E={E[-1]:.4g} P={P[-1]:.4g} → {classify(E, P)}")
88
89     # TODO: also produce a plot of E vs window and P vs window (CT signals).
90     plt.show()
91
92
93 if __name__ == "__main__":
94     main()

```

Reference solution

The energy of $e^{-|t|}$ is $\int_{-\infty}^{\infty} e^{-2|t|} dt = 1$, so it is an **energy** signal (finite energy, power $\rightarrow 0$). A sinusoid $\cos(\omega t)$ has $\langle \cos^2 \rangle = \frac{1}{2}$, so its energy grows without bound but its power tends to $\frac{1}{2}$: a **power** signal; the constant 1 is likewise a power signal with $P = 1$. The ramps grow, so both energy and power blow up: **neither**. For $(0.8)^n u[n]$ the energy is the geometric sum $\sum (0.8)^{2n} = 1/(1 - 0.64) = 25/9 \approx 2.78$, an **energy** signal. The trick in `classify` is to decide “finite” vs. “infinite” from the *trend* across growing windows rather than from a single window.

```

1  """
2  Problem 1 - Energy, Power, and Signal Classification
3  Reference solution.
4  """
5  import numpy as np
6  import matplotlib
7  import matplotlib.pyplot as plt
8
9  # =====
10 # ANSWER IMPLEMENTATION
11 # =====
12
13 def energy_ct(t: np.ndarray, x: np.ndarray) → float:
14     """Energy of a sampled continuous-time signal: integral of |x|^2 dt
15     (trapezoidal rule)."""
16     return float(np.trapezoid(np.abs(x) ** 2, t))
17
18
19 def power_ct(t: np.ndarray, x: np.ndarray) → float:
20     """Average power of a sampled CT signal: energy / (length of interval)."""
21     T = t[-1] - t[0]
22     return energy_ct(t, x) / T
23
24
25 def energy_dt(x: np.ndarray) → float:
26     """Energy of a discrete-time signal: sum of |x[n]|^2."""
27     return float(np.sum(np.abs(x) ** 2))
28
29
30 def power_dt(x: np.ndarray) → float:
31     """Average power of a discrete-time signal: mean of |x[n]|^2."""
32     return float(np.mean(np.abs(x) ** 2))
33
34
35 def classify(energies: np.ndarray, powers: np.ndarray,
36             e_tol: float = 1e6, p_tol: float = 1e-6) → str:
37     """Classify a signal from its energy/power measured over a sequence of
38     growing windows.
39
40     * Energy signal : energy converges to a finite limit, power → 0.
41     * Power signal  : energy grows without bound, but power → finite > 0.
42     * Neither       : both blow up (or neither settles).
43     """
44     E_last = energies[-1]
45     P_last = powers[-1]
46     # Did the energy settle (last two windows agree to within 1%)?
47     energy_settled = abs(energies[-1] - energies[-2]) <= 0.02 * abs(energies[-1] + 1e-12)
48     # Did the power settle to a positive constant?
49     power_settled = abs(powers[-1] - powers[-2]) <= 0.02 * abs(powers[-1] + 1e-12)
50
51     if energy_settled and E_last < e_tol:
52         return "energy"
53     if power_settled and P_last > p_tol:
54         return "power"
55     return "neither"
56
57
58 # -----
59 # Test signals (provided)
60 # -----
61 def make_ct_signals():
62     """Return a dict name → callable x(t) for CT signals."""

```

```

63     return {
64         "e^{-|t|}":          lambda t: np.exp(-np.abs(t)),
65         "cos(2*pi*0.5*t)":   lambda t: np.cos(2 * np.pi * 0.5 * t),
66         "constant 1":       lambda t: np.ones_like(t),
67         "t (ramp)":         lambda t: t.astype(float),
68     }
69
70
71 def make_dt_signals():
72     return {
73         "(0.8)^n u[n]":      lambda n: np.where(n >= 0, 0.8 ** np.abs(n), 0.0),
74         "cos(pi n / 6)":     lambda n: np.cos(np.pi * n / 6),
75         "n (ramp)":         lambda n: n.astype(float),
76     }
77
78
79 def analyse_ct(name, xfunc, half_widths):
80     Es, Ps = [], []
81     for T in half_widths:
82         t = np.linspace(-T, T, int(2000 * T) + 1)
83         x = xfunc(t)
84         Es.append(energy_ct(t, x))
85         Ps.append(power_ct(t, x))
86     return np.array(Es), np.array(Ps)
87
88
89 def analyse_dt(name, xfunc, half_lengths):
90     Es, Ps = [], []
91     for M in half_lengths:
92         n = np.arange(-M, M + 1)
93         x = xfunc(n)
94         Es.append(energy_dt(x))
95         Ps.append(power_dt(x))
96     return np.array(Es), np.array(Ps)
97
98
99 def main():
100     half_widths = [5, 10, 20, 40, 80]
101     print("=== Continuous-time signals ===")
102     for name, f in make_ct_signals().items():
103         E, P = analyse_ct(name, f, half_widths)
104         kind = classify(E, P)
105         print(f"{name:22s} | E(last)={E[-1]:.4g} P(last)={P[-1]:.4g} → {kind}")
106
107     print("\n=== Discrete-time signals ===")
108     half_lengths = [24, 120, 600, 1200, 3000]
109     for name, f in make_dt_signals().items():
110         E, P = analyse_dt(name, f, half_lengths)
111         kind = classify(E, P)
112         print(f"{name:22s} | E(last)={E[-1]:.4g} P(last)={P[-1]:.4g} → {kind}")
113
114     # Figure: energy growth for one of each type
115     fig, ax = plt.subplots(1, 2, figsize=(10, 4))
116     for name, f in make_ct_signals().items():
117         E, P = analyse_ct(name, f, half_widths)
118         ax[0].plot(half_widths, E, marker="o", label=name)
119         ax[0].set_title("Energy vs window (CT)")
120         ax[0].set_xlabel("half-width T"); ax[0].set_ylabel("E[-T,T]")
121         ax[0].set_ylim(0, 20); ax[0].legend(fontsize=7); ax[0].grid(alpha=.3)
122     for name, f in make_ct_signals().items():
123         E, P = analyse_ct(name, f, half_widths)
124         ax[1].plot(half_widths, P, marker="s", label=name)
125         ax[1].set_title("Power vs window (CT)")
126         ax[1].set_xlabel("half-width T"); ax[1].set_ylabel("P[-T,T]")
127         ax[1].legend(fontsize=7); ax[1].grid(alpha=.3)
128     fig.tight_layout()
129     plt.show()
130
131     # ---- verification ----
132     t = np.linspace(-60, 60, 240001)
133     assert abs(energy_ct(t, np.exp(-np.abs(t))) - 1.0) < 1e-3, "E[e^{-|t|}] should be 1"
134     n = np.arange(0, 200)
135     assert abs(energy_dt(0.8 ** n) - 1/(1-0.64)) < 1e-6, "geometric energy"
136     print("\nAll assertions passed.")
137
138

```

```

139 if __name__ == "__main__":
140     main()

```

Expected console output:

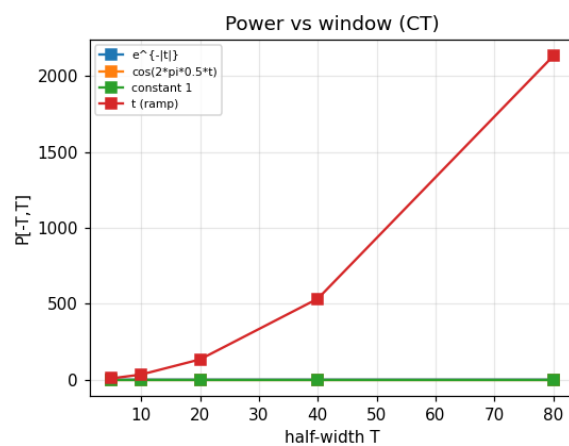
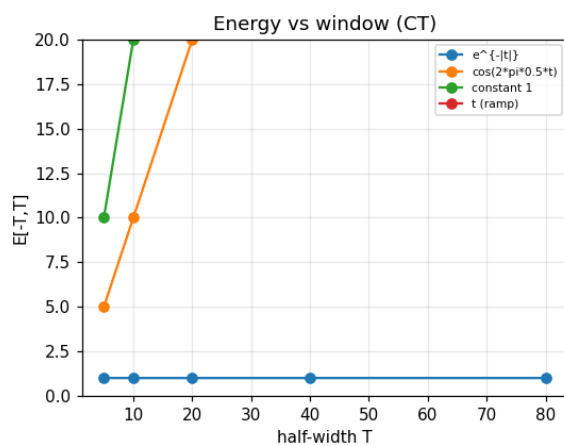
```

=== Continuous-time signals ===
e^{-|t|}          | E(last)=1   P(last)=0.00625  -> energy
cos(2*pi*0.5*t)   | E(last)=80  P(last)=0.5      -> power
constant 1        | E(last)=160 P(last)=1        -> power
t (ramp)          | E(last)=3.413e+05 P(last)=2133 -> neither

=== Discrete-time signals ===
(0.8)^n u[n]      | E(last)=2.778 P(last)=0.0004629 -> energy
cos(pi n / 6)     | E(last)=3001 P(last)=0.5001     -> power
n (ramp)          | E(last)=1.801e+10 P(last)=3.001e+06 -> neither

All assertions passed.

```



Problem 2: The General Affine Transform $x(\alpha t + \beta)$ (Marks: 10 Suggested time: 30 minutes)

Lecture 1 gives a three-step recipe to obtain $x(\alpha t + \beta)$ from $x(t)$: first shift, then reverse the time axis only if $\alpha < 0$, then scale time by $|\alpha|$:

$$x(t) \xrightarrow{\text{shift } \beta} x(t + \beta) \xrightarrow{\text{reverse if } \alpha < 0} \xrightarrow{\text{scale } |\alpha|} x(\alpha t + \beta).$$

You will implement this recipe for a *sampled* continuous-time signal. Because the samples land on a fixed grid, each elementary operation must *resample* the signal by linear interpolation.

Tasks.

1. Implement `interp_sample(t, x, t_query)` (linear interpolation, zero outside the original support).
2. Implement the three elementary operations `apply_shift`, `apply_reverse`, `apply_scale`, each returning the transformed signal *on the same grid* `t`. Note the lecture convention: $x(t + \beta)$ with $\beta > 0$ shifts the graph to the *left*.
3. Implement `affine_transform(t, x, alpha, beta)` that composes the three steps in the correct order.
4. Verify against a direct evaluation $x(\alpha t + \beta)$ for the four cases $(\alpha, \beta) \in \{(-3, 2), (2, 3), (0.5, -1), (-1, 0)\}$ by reporting the mean-squared error, and plot all four. The case $(-3, 2)$ is the worked example from the slides.

Marks: interpolation 2, three elementary operations 3, recipe composition 3, verification and plots 2.

Provided template (fill in the functions marked ANSWER IMPLEMENTATION)

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  T_MIN, T_MAX, N = -6.0, 6.0, 4801
5
6
7  def x_of_t(t: np.ndarray) → np.ndarray:
8      """Base signal (given)."""
9      y = np.zeros_like(t, dtype=float)
10     m1 = (t >= 0) & (t < 1); y[m1] = t[m1]
11     m2 = (t >= 1) & (t <= 2); y[m2] = 2 - t[m2]
12     return y
13
14
15 # =====
16 # ANSWER IMPLEMENTATION
17 # =====
18
19 def interp_sample(t, x, t_query):
20     """Evaluate sampled x on grid t at arbitrary t_query via linear
21     interpolation; 0 outside the original range."""
22     raise NotImplementedError
23
24
25 def apply_shift(t, x, beta):
26     """Return samples of x(t + beta) on the SAME grid t (beta>0 shifts LEFT)."""
27     raise NotImplementedError
28
29
30 def apply_reverse(t, x):
31     """Return samples of x(-t) on grid t."""
32     raise NotImplementedError
33

```

```

34
35 def apply_scale(t, x, a):
36     """Return samples of  $x(at)$  on grid  $t$ ,  $a>0$ ."""
37     raise NotImplementedError
38
39
40 def affine_transform(t, x, alpha, beta):
41     """ $y(t)=x(\alpha t+\beta)$  via recipe: shift by  $\beta \rightarrow$  reverse if  $\alpha<0$ 
42      $\rightarrow$  scale by  $|\alpha|$ ."""
43     raise NotImplementedError
44
45
46 def direct_transform(t, x, alpha, beta):
47     """Ground truth: evaluate  $x$  directly at  $\alpha t + \beta$  (given, uses your
48     interp_sample)."""
49     return interp_sample(t, x, alpha * t + beta)
50
51
52 def plot_case(t, x, y_recipe, y_direct, alpha, beta, ax):
53     ax.plot(t, x, "k--", lw=1, label="x(t)")
54     ax.plot(t, y_recipe, lw=2, label="recipe")
55     ax.plot(t, y_direct, ":", lw=2, label="direct")
56     ax.set_title(fr"$y(t)=x(\alpha t+\beta)$")
57     ax.grid(alpha=.3); ax.legend(fontsize=7)
58
59
60 def main():
61     t = np.linspace(T_MIN, T_MAX, N)
62     x = x_of_t(t)
63     cases = [(-3.0, 2.0), (2.0, 3.0), (0.5, -1.0), (-1.0, 0.0)]
64     fig, axes = plt.subplots(2, 2, figsize=(10, 7))
65     for (alpha, beta), ax in zip(cases, axes.ravel()):
66         yr = affine_transform(t, x, alpha, beta)
67         yd = direct_transform(t, x, alpha, beta)
68         print(f"alpha={alpha:+g}, beta={beta:+g}  $\rightarrow$  MSE={np.mean((yr-yd)**2):.2e}")
69         plot_case(t, x, yr, yd, alpha, beta, ax)
70     fig.tight_layout(); plt.show()
71
72
73 if __name__ == "__main__":
74     main()

```

Reference solution

The key subtlety is order. To build $x(\alpha t + \beta)$ you must apply the shift to the *original* signal first; only then reverse (if needed) and compress. If you scale first and shift afterwards you get $x(\alpha(t + \beta)) = x(\alpha t + \alpha\beta)$, a different signal — exactly the “changing the order gives a different answer” slide. Numerically, each operation is a resample: $x(t + \beta)$ is the original samples read at the query points $t + \beta$, $x(-t)$ reads at $-t$, and $x(\alpha t)$ reads at αt . With correct composition the recipe matches the direct evaluation to machine precision (MSE = 0).

```

1  """
2  Problem 2 - General affine time transform  $y(t) = x(\alpha t + \beta)$ 
3  via the lecture's shift  $\rightarrow$  reverse  $\rightarrow$  scale recipe.
4  Reference solution.
5  """
6  import numpy as np
7  import matplotlib
8  import matplotlib.pyplot as plt
9
10 T_MIN, T_MAX, N = -6.0, 6.0, 4801
11
12
13 def x_of_t(t: np.ndarray)  $\rightarrow$  np.ndarray:
14     """Base signal: the piecewise 'triangle-with-a-flat-top' used in lecture,
15     slightly enriched so that reversal and shifting are visible."""
16     y = np.zeros_like(t, dtype=float)
17     m1 = (t >= 0) & (t < 1) # rising ramp
18     y[m1] = t[m1]

```



```

19     m2 = (t >= 1) & (t <= 2)          # flat top then fall
20     y[m2] = 2 - t[m2]
21     return y
22
23
24 # =====
25 # ANSWER IMPLEMENTATION
26 # =====
27
28 def interp_sample(t: np.ndarray, x: np.ndarray, t_query: np.ndarray) → np.ndarray:
29     """Evaluate the sampled signal x (defined on grid t) at arbitrary query
30     times using linear interpolation. Return 0 outside the original range."""
31     return np.interp(t_query, t, x, left=0.0, right=0.0)
32
33
34 def apply_shift(t: np.ndarray, x: np.ndarray, beta: float) → np.ndarray:
35     """Return samples of x(t + beta) on the SAME grid t.
36     beta > 0 shifts the graph LEFT (advance)."""
37     return interp_sample(t, x, t + beta)
38
39
40 def apply_reverse(t: np.ndarray, x: np.ndarray) → np.ndarray:
41     """Return samples of x(-t) on the same grid t."""
42     return interp_sample(t, x, -t)
43
44
45 def apply_scale(t: np.ndarray, x: np.ndarray, a: float) → np.ndarray:
46     """Return samples of x(a*t) on the same grid t. Require a > 0
47     (reversal handles the sign separately)."""
48     assert a > 0
49     return interp_sample(t, x, a * t)
50
51
52 def affine_transform(t: np.ndarray, x: np.ndarray,
53                     alpha: float, beta: float) → np.ndarray:
54     """Produce y(t) = x(alpha*t + beta) using the 3-step recipe:
55     Step 1 g1(t) = x(t + beta)                (shift)
56     Step 2 g2(t) = g1(-t) only if alpha < 0    (reversal)
57     Step 3 g3(t) = g2(|alpha| t)              (scale)
58     """
59     g1 = apply_shift(t, x, beta)
60     g2 = apply_reverse(t, g1) if alpha < 0 else g1
61     g3 = apply_scale(t, g2, abs(alpha))
62     return g3
63
64
65 def direct_transform(t: np.ndarray, x: np.ndarray,
66                    alpha: float, beta: float) → np.ndarray:
67     """Ground truth: evaluate x directly at alpha*t + beta."""
68     return interp_sample(t, x, alpha * t + beta)
69
70
71 # -----
72 # Provided plotting
73 # -----
74 def plot_case(t, x, y_recipe, y_direct, alpha, beta, ax):
75     ax.plot(t, x, "k--", lw=1, label="x(t)")
76     ax.plot(t, y_recipe, lw=2, label="recipe")
77     ax.plot(t, y_direct, ":", lw=2, label="direct")
78     ax.set_title(fr"$y(t)=x\{alpha:g\}t\{beta:g\}$")
79     ax.grid(alpha=.3); ax.legend(fontsize=7)
80
81
82 def main():
83     t = np.linspace(T_MIN, T_MAX, N)
84     x = x_of_t(t)
85
86     cases = [(-3.0, 2.0), (2.0, 3.0), (0.5, -1.0), (-1.0, 0.0)]
87     fig, axes = plt.subplots(2, 2, figsize=(10, 7))
88     for (alpha, beta), ax in zip(cases, axes.ravel()):
89         yr = affine_transform(t, x, alpha, beta)
90         yd = direct_transform(t, x, alpha, beta)
91         mse = float(np.mean((yr - yd) ** 2))
92         print(f"alpha={alpha:+g}, beta={beta:+g} → MSE(recipe,direct)={mse:.2e}")
93         plot_case(t, x, yr, yd, alpha, beta, ax)
94     fig.tight_layout()

```

```

95 plt.show()
96
97 # ---- verification: recipe must match direct evaluation ----
98 for alpha, beta in cases:
99     yr = affine_transform(t, x, alpha, beta)
100     yd = direct_transform(t, x, alpha, beta)
101     assert float(np.mean((yr - yd) ** 2)) < 1e-4, (alpha, beta)
102     print("All assertions passed.")
103
104
105 if __name__ == "__main__":
106     main()

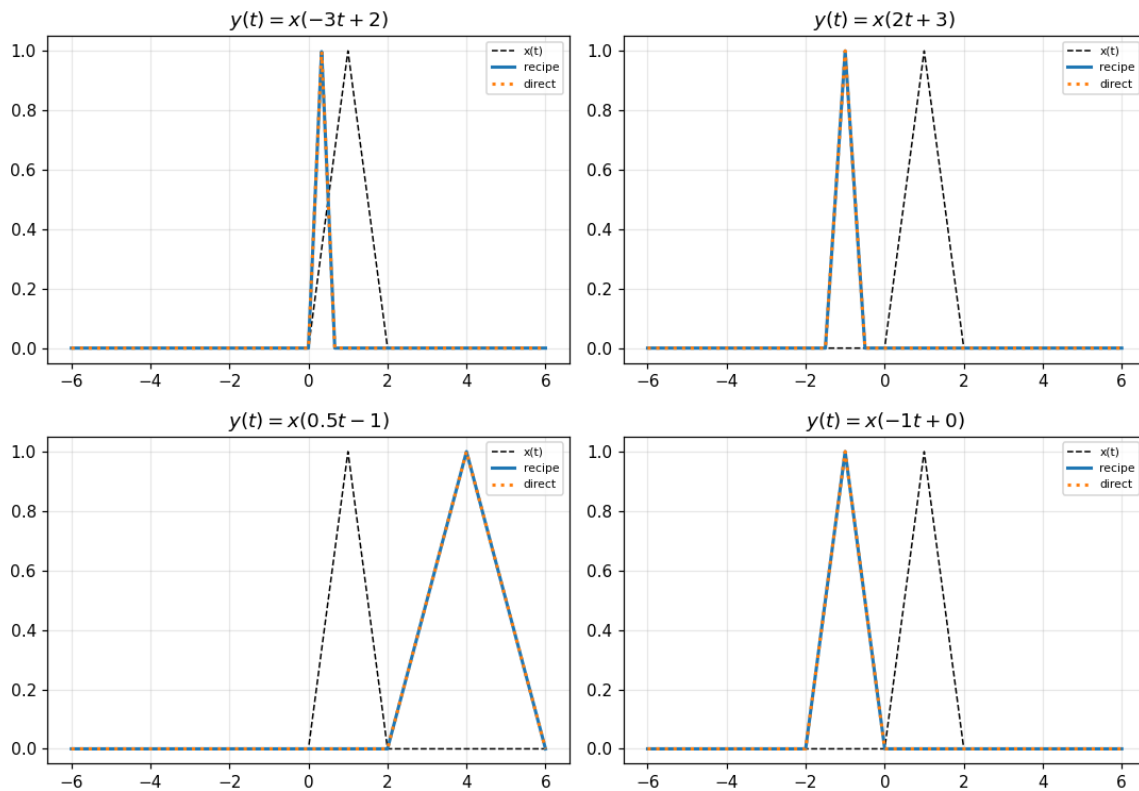
```

Expected console output:

```

alpha=-3, beta=+2 -> MSE(recipe,direct)=0.00e+00
alpha=+2, beta=+3 -> MSE(recipe,direct)=0.00e+00
alpha=+0.5, beta=-1 -> MSE(recipe,direct)=0.00e+00
alpha=-1, beta=+0 -> MSE(recipe,direct)=0.00e+00
All assertions passed.

```



Problem 3: Discrete-Time Periodicity and Fundamental Period (Marks: 10)

Suggested time: 28 minutes)

Unlike its continuous cousin, a discrete sinusoid $\cos(\Omega_0 n + \phi)$ is periodic *only* when $\Omega_0/2\pi$ is a rational number k/N ; then the signal repeats with some period, and the *fundamental* period is $N/\gcd(k, N)$. A sum of sinusoids is periodic iff each term is, and its fundamental period is the least common multiple of the individual periods.

Tasks.

1. Implement `sinusoid_period(Omega)` returning `(is_periodic, N)`. Detect rationality of $\Omega/2\pi$ using `Fraction(...).limit_denominator`, and confirm the rational approximation is faithful before trusting it.
2. Implement `combined_period(omegas)` (LCM of the individual fundamental periods).
3. Implement `verify_period(x_func, N)` that numerically confirms $x[n + N] = x[n]$ by returning the maximum residual over a test range.
4. Report results for the single frequencies $\pi/4$, $3\pi/5$, $\pi/3$, 1, $2\pi/7$, and for the sum $\cos(\pi n/3) + \frac{1}{2}\sin(\pi n/4)$ (whose period should be $\text{lcm}(6, 8) = 24$). Plot the periodic sum over two periods and mark the period boundaries.

Marks: `sinusoid_period` 4, `combined_period` 2, `verify_period` 2, correct answers and plot 2.

Provided template (fill in the functions marked ANSWER IMPLEMENTATION)

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from fractions import Fraction
4 from math import gcd
5
6 # =====
7 # ANSWER IMPLEMENTATION
8 # =====
9
10 def sinusoid_period(Omega: float, max_den: int = 1000):
11     """cos(Omega*n) is periodic iff Omega/(2*pi) is rational = k/N. Return
12     (is_periodic, N_fundamental) or (False, None). Hint: fractions.Fraction
13     with .limit_denominator, then divide N by gcd(k, N)."""
14     raise NotImplementedError
15
16
17 def lcm(a: int, b: int) -> int:
18     return a * b // gcd(a, b)
19
20
21 def combined_period(omegas):
22     """Fundamental period of a sum of sinusoids = LCM of individual periods.
23     Return (is_periodic, N)."""
24     raise NotImplementedError
25
26
27 def verify_period(x_func, N: int, n_test=None) -> float:
28     """Return max |x[n+N]-x[n]| over a test range (small => period N)."""
29     raise NotImplementedError
30
31
32 def stem(ax, n, x, label):
33     m1, s1, b1 = ax.stem(n, x, label=label); b1.set_visible(False)
34     ax.grid(alpha=.3); ax.set_xlabel("n"); ax.set_ylabel("amp")
35
36
37 def main():
38     print("=== Single sinusoids ===")
39     for w in [np.pi/4, 3*np.pi/5, np.pi/3, 1.0, 2*np.pi/7]:

```

```

40     ok, N = sinusoid_period(w)
41     if ok:
42         r = verify_period(lambda n: np.cos(w*n), N)
43         print(f"Omega={w:.5f} → periodic, N={N:3d} (residual={r:.1e})")
44     else:
45         print(f"Omega={w:.5f} → NOT periodic")
46
47     print("\n=== Sum of sinusoids ===")
48     omegas = [np.pi/3, np.pi/4]
49     ok, N = combined_period(omegas)
50     def xsum(n): return np.cos(np.pi*n/3) + 0.5*np.sin(np.pi*n/4)
51     print(f"omegas={omegas} → periodic={ok}, N={N}")
52
53     if ok:
54         n = np.arange(0, 2*N)
55         fig, ax = plt.subplots(figsize=(9, 3.2))
56         stem(ax, n, xsum(n), "x[n]")
57         ax.set_title(f"Fundamental period N={N}"); ax.legend(fontsize=8)
58         fig.tight_layout()
59     plt.show()
60
61
62 if __name__ == "__main__":
63     main()

```

Reference solution

$\Omega = 1$ gives $\Omega/2\pi = 1/2\pi$, which is irrational: the signal is *not* periodic, even though the underlying continuous $\cos(t)$ is. For $\Omega = 3\pi/5$ we have $\Omega/2\pi = 3/10$, so $N = 10$; for $\Omega = \pi/4$, $\Omega/2\pi = 1/8$, so $N = 8$. The residual check returns numbers at the level of floating-point round-off ($\sim 10^{-15}$), confirming exact periodicity. The sum inherits $N = \text{lcm}(6, 8) = 24$; mixing a rational with an irrational frequency destroys periodicity entirely.

```

1  """
2  Problem 3 - Discrete-time periodicity: is  $x[n]$  periodic, and if so what is
3  its fundamental period  $N$ ?
4  Reference solution.
5  """
6  import numpy as np
7  import matplotlib
8  import matplotlib.pyplot as plt
9  from fractions import Fraction
10 from math import gcd
11
12
13 # =====
14 # ANSWER IMPLEMENTATION
15 # =====
16
17 def sinusoid_period(Omega: float, max_den: int = 1000):
18     """A discrete sinusoid  $\cos(\Omega n + \phi)$  is periodic iff  $\Omega/(2\pi)$ 
19     is rational, i.e.  $\Omega = 2\pi \cdot (k/N)$ . Then the (not necessarily
20     fundamental) period is  $N$ , and the FUNDAMENTAL period is  $N/\text{gcd}(k, N)$ .
21
22     Returns (is_periodic, N) with N an int, or (False, None).
23     """
24     ratio = Omega / (2 * np.pi)
25     frac = Fraction(ratio).limit_denominator(max_den)
26     # Check the rational approximation is actually faithful.
27     if abs(float(frac) - ratio) > 1e-9:
28         return False, None
29     k, N = frac.numerator, frac.denominator
30     if N == 0:
31         return False, None
32     N_fund = N // gcd(abs(k), N) if k != 0 else 1
33     return True, N_fund
34
35
36 def lcm(a: int, b: int) → int:

```

```

37     return a * b // gcd(a, b)
38
39
40 def combined_period(omegas):
41     """Fundamental period of a sum of sinusoids = LCM of the individual
42     fundamental periods. Returns (is_periodic, N)."""
43     periods = []
44     for w in omegas:
45         ok, N = sinusoid_period(w)
46         if not ok:
47             return False, None
48         periods.append(N)
49     N = 1
50     for p in periods:
51         N = lcm(N, p)
52     return True, N
53
54
55 def verify_period(x_func, N: int, n_test=None) → float:
56     """Numerically verify periodicity: max |x[n+N] - x[n]| over a test range.
57     A small value confirms period N."""
58     if n_test is None:
59         n_test = np.arange(-50, 51)
60     return float(np.max(np.abs(x_func(n_test + N) - x_func(n_test))))
61
62
63 # -----
64 # Provided helpers
65 # -----
66 def stem(ax, n, x, label):
67     ml, sl, bl = ax.stem(n, x, label=label)
68     bl.set_visible(False)
69     ax.grid(alpha=.3); ax.set_xlabel("n"); ax.set_ylabel("amp")
70
71
72 def main():
73     print("=== Single sinusoids ===")
74     tests = [np.pi / 4, 3 * np.pi / 5, np.pi / 3, 1.0, 2 * np.pi / 7]
75     for w in tests:
76         ok, N = sinusoid_period(w)
77         if ok:
78             resid = verify_period(lambda n: np.cos(w * n), N)
79             print(f"Omega={w:.5f} → periodic, N={N:3d} (residual={resid:.1e})")
80         else:
81             print(f"Omega={w:.5f} → NOT periodic")
82
83     print("\n=== Sum of sinusoids ===")
84     #  $x[n] = \cos(\pi n/3) + 0.5 \sin(\pi n/4) \rightarrow N = \text{lcm}(6, 8) = 24$ 
85     omegas = [np.pi / 3, np.pi / 4]
86     ok, N = combined_period(omegas)
87
88     def xsum(n):
89         return np.cos(np.pi * n / 3) + 0.5 * np.sin(np.pi * n / 4)
90
91     resid = verify_period(xsum, N) if ok else None
92     print(f"omegas={omegas} → periodic={ok}, N={N}, residual={resid:.1e}")
93
94     # A non-periodic sum: mixing a rational and an irrational frequency
95     ok2, N2 = combined_period([np.pi / 3, 1.0])
96     print(f"omegas=[pi/3, 1.0] → periodic={ok2}, N={N2}")
97
98     # Plot the periodic sum over ~2 periods
99     n = np.arange(0, 2 * N)
100     fig, ax = plt.subplots(figsize=(9, 3.2))
101     stem(ax, n, xsum(n), "x[n]=cos(pi n/3)+0.5 sin(pi n/4)")
102     for m in range(0, 2 * N + 1, N):
103         ax.axvline(m - 0.5, color="r", ls="--", alpha=.4)
104     ax.set_title(f"Fundamental period N = {N}")
105     ax.legend(fontsize=8)
106     fig.tight_layout()
107     plt.show()
108
109     # --- verification ---
110     assert sinusoid_period(np.pi / 4) == (True, 8)
111     assert sinusoid_period(3 * np.pi / 5) == (True, 10)
112     assert sinusoid_period(np.pi / 3) == (True, 6)

```

```

113     assert sinusoid_period(1.0)[0] is False
114     assert combined_period([np.pi / 3, np.pi / 4]) == (True, 24)
115     print("\nAll assertions passed.")
116
117
118 if __name__ == "__main__":
119     main()

```

Expected console output:

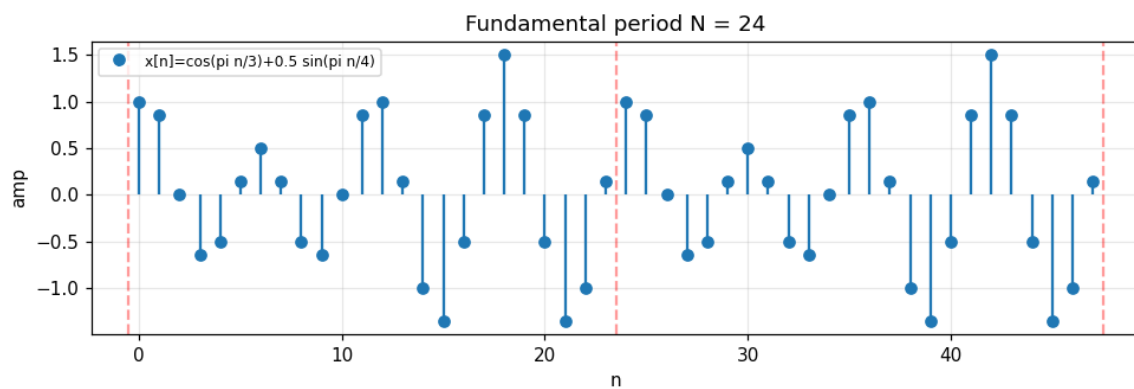
```

=== Single sinusoids ===
Omega=0.78540 -> periodic, N= 8 (residual=2.8e-15)
Omega=1.88496 -> periodic, N= 10 (residual=7.4e-15)
Omega=1.04720 -> periodic, N= 6 (residual=3.3e-15)
Omega=1.00000 -> NOT periodic
Omega=0.89760 -> periodic, N= 7 (residual=2.5e-16)

=== Sum of sinusoids ===
omegas=[1.0471975511965976, 0.7853981633974483] -> periodic=True, N=24, residual=8.5e-15
omegas=[pi/3, 1.0] -> periodic=False, N=None

All assertions passed.

```



Problem 4: Can an Infinite-Range Signal Have Finite Energy? (Marks: 10)

Suggested time: 28 minutes)

The lecture showed a signal that is nonzero for all t yet has energy exactly 4. Here you will investigate this convergence numerically by accumulating energy over windows $[-T, T]$ of growing half-width T and deciding whether the total energy is finite or infinite.

Tasks.

1. Implement `energy_window(xfunc, T)` (trapezoidal integration of $|x|^2$ over $[-T, T]$); cap the sample count so large windows stay fast.
2. Implement `energy_curve` and `estimated_limit`.
3. Implement `classify_convergence(Ts, E)` assuming the window half-widths grow *geometrically* (they double). The correct test is not whether E is still changing but whether its *increments* are dying out: return "finite" if the increments at least halve over two doubling steps, else "infinite".
4. Run on: $e^{-|t|}$ (energy 1), the lecture signal (1 for $|t| \leq 1$, $1/|t|$ otherwise) (energy 4), $1/\sqrt{1+|t|}$, and $\cos t$. Plot all four $E(T)$ curves.

Marks: `energy_window` 2, `energy_curve` 1, `classify_convergence` 4, limit + plot + correct classifications 3.

Provided template (fill in the functions marked ANSWER IMPLEMENTATION)

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # =====
5  # ANSWER IMPLEMENTATION
6  # =====
7
8  def energy_window(xfunc, T: float, samples_per_unit: int = 800) → float:
9      """Numerically integrate |x(t)|^2 over [-T, T] (trapezoidal). Cap the
10         number of samples so large T stays fast."""
11      raise NotImplementedError
12
13
14  def energy_curve(xfunc, Ts):
15      """Return array E(T) for each half-width in Ts."""
16      raise NotImplementedError
17
18
19  def classify_convergence(Ts, E) → str:
20      """Assume Ts grows geometrically (doubles). Return "finite" if the
21         increments dE_k = E[k+1]-E[k] shrink toward 0 (finite energy), else
22         "infinite"."""
23      raise NotImplementedError
24
25
26  def estimated_limit(Ts, E) → float:
27      """Best estimate of total energy = largest-window value."""
28      raise NotImplementedError
29
30
31  # -----
32  # Test signals (given)
33  # -----
34  def x_exp(t):     return np.exp(-np.abs(t))
35  def x_slow(t):    return 1.0 / np.sqrt(1.0 + np.abs(t))
36  def x_cos(t):     return np.cos(t)
37  def x_tail(t):
38      y = np.zeros_like(t, dtype=float); inside = np.abs(t) <= 1
39      y[inside] = 1.0; y[~inside] = 1.0 / np.abs(t[~inside]); return y
40
41

```

```

42 def main():
43     Ts = 2.0 ** np.arange(1, 13) # 2, 4, ..., 4096
44     signals = {"e^{-|t|}": x_exp, "lecture 1/|t|": x_tail,
45               "1/sqrt(1+|t|)": x_slow, "cos(t)": x_cos}
46     fig, ax = plt.subplots(figsize=(9, 4.2))
47     for name, f in signals.items():
48         E = energy_curve(f, Ts)
49         kind = classify_convergence(Ts, E)
50         lim = estimated_limit(Ts, E) if kind == "finite" else np.inf
51         print(f"{name:18s} → {kind:8s} E(inf) ~ {lim:.4g}")
52         ax.plot(Ts, E, marker="o", label=f"{name} [{kind}]")
53     ax.set_xscale("log"); ax.set_ylim(0, 12); ax.grid(alpha=.3)
54     ax.set_xlabel("T"); ax.set_ylabel("E[-T,T]"); ax.legend(fontsize=8)
55     fig.tight_layout(); plt.show()
56
57
58 if __name__ == "__main__":
59     main()

```

Reference solution

The lecture $1/|t|$ signal converges to 4 but *slowly*: the tail beyond T contributes $\approx 2/T$, so $E(T) = 4 - 2/T$. A naive “has E stopped changing?” test with a fixed absolute threshold would wrongly call it divergent. The increment test fixes this: with geometric windows the increments themselves shrink like $1/T$, halving each step, which flags **finite**. By contrast $1/\sqrt{1+|t|}$ has $|x|^2 = 1/(1+|t|)$, whose integral grows like $\ln T$ — the increments stay roughly constant, so it is **infinite**; and $\cos t$ accumulates $\sim T$, increments growing: **infinite** (a power signal).

```

1  """
2  Problem 4 - Does an infinite-range signal have finite energy?
3  Study  $E(T) = \int_{-T}^T |x(t)|^2 dt$  as  $T$  grows.
4  Reference solution.
5  """
6  import numpy as np
7  import matplotlib
8  import matplotlib.pyplot as plt
9
10
11 # =====
12 # ANSWER IMPLEMENTATION
13 # =====
14
15 def energy_window(xfunc, T: float, samples_per_unit: int = 800) → float:
16     """Numerically integrate  $|x(t)|^2$  over  $[-T, T]$  (trapezoidal)."""
17     n = min(4_000_001, int(samples_per_unit * T) + 1)
18     t = np.linspace(-T, T, n)
19     x = xfunc(t)
20     return float(np.trapezoid(np.abs(x) ** 2, t))
21
22
23 def energy_curve(xfunc, Ts):
24     """Return array  $E(T)$  for each window half-width in Ts."""
25     return np.array([energy_window(xfunc, T) for T in Ts])
26
27
28 def classify_convergence(Ts, E) → str:
29     """Decide from the tail of  $E(T)$ , assuming Ts grows geometrically (e.g. it
30     doubles each step). The right question is not whether E is still changing
31     but whether the *increments* are dying out.
32
33      $dE_k = E(T_{k+1}) - E(T_k)$ 
34
35     * finite energy :  $dE_k$  shrinks toward 0 (here: at least halves over
36     two doubling steps).
37     * otherwise      :  $dE_k$  stays roughly constant (log-divergent) or grows
38     (power signal) → infinite energy.
39     """
40     dE = np.diff(E)
41     if len(dE) < 3:

```



```

42     raise ValueError("need at least 4 window sizes")
43     if dE[-1] <= 0.6 * dE[-3] + 1e-12:
44         return "finite"
45     return "infinite"
46
47
48 def estimated_limit(Ts, E) → float:
49     """Best estimate of the total energy: the last (largest-window) value."""
50     return float(E[-1])
51
52
53 # -----
54 # Test signals (provided)
55 # -----
56 def x_exp(t):          # two-sided decaying exponential ; E = 1/a with a=1
57     return np.exp(-np.abs(t))
58
59
60 def x_tail(t):          # the lecture signal: 1 on |t|≤1, then 1/|t| ; E = 4
61     y = np.zeros_like(t, dtype=float)
62     inside = np.abs(t) <= 1
63     y[inside] = 1.0
64     outside = ~inside
65     y[outside] = 1.0 / np.abs(t[outside])
66     return y
67
68
69 def x_slow(t):          # 1/sqrt(1+|t|) : tail ~ 1/|t| under the square → diverges
70     return 1.0 / np.sqrt(1.0 + np.abs(t))
71
72
73 def x_cos(t):           # cos(t): power signal, energy diverges linearly
74     return np.cos(t)
75
76
77 def main():
78     Ts = 2.0 ** np.arange(1, 13)    # 2, 4, 8, ..., 4096 (geometric)
79     signals = {
80         "e^{-|t|} (E=1)":      x_exp,
81         "lecture 1/|t| (E=4)": x_tail,
82         "1/sqrt(1+|t|)":       x_slow,
83         "cos(t)":              x_cos,
84     }
85
86     fig, ax = plt.subplots(figsize=(9, 4.2))
87     for name, f in signals.items():
88         E = energy_curve(f, Ts)
89         kind = classify_convergence(Ts, E)
90         lim = estimated_limit(Ts, E) if kind == "finite" else np.inf
91         print(f"{name:22s} → {kind:8s} E(inf) ~ {lim:.4g}")
92         ax.plot(Ts, E, marker="o", label=f"{name} [{kind}]")
93     ax.set_xscale("log")
94     ax.set_xlabel("window half-width T"); ax.set_ylabel("E over [-T, T]")
95     ax.set_ylim(0, 12); ax.grid(alpha=.3); ax.legend(fontsize=8)
96     ax.set_title("Energy accumulated vs window size")
97     fig.tight_layout()
98     plt.show()
99
100 # ---- verification ----
101 assert classify_convergence(Ts, energy_curve(x_exp, Ts)) == "finite"
102 assert abs(estimated_limit(Ts, energy_curve(x_exp, Ts)) - 1.0) < 1e-2
103 assert classify_convergence(Ts, energy_curve(x_tail, Ts)) == "finite"
104 assert abs(estimated_limit(Ts, energy_curve(x_tail, Ts)) - 4.0) < 1e-2
105 assert classify_convergence(Ts, energy_curve(x_slow, Ts)) == "infinite"
106 assert classify_convergence(Ts, energy_curve(x_cos, Ts)) == "infinite"
107 print("All assertions passed.")
108
109
110 if __name__ == "__main__":
111     main()

```

Expected console output:

```

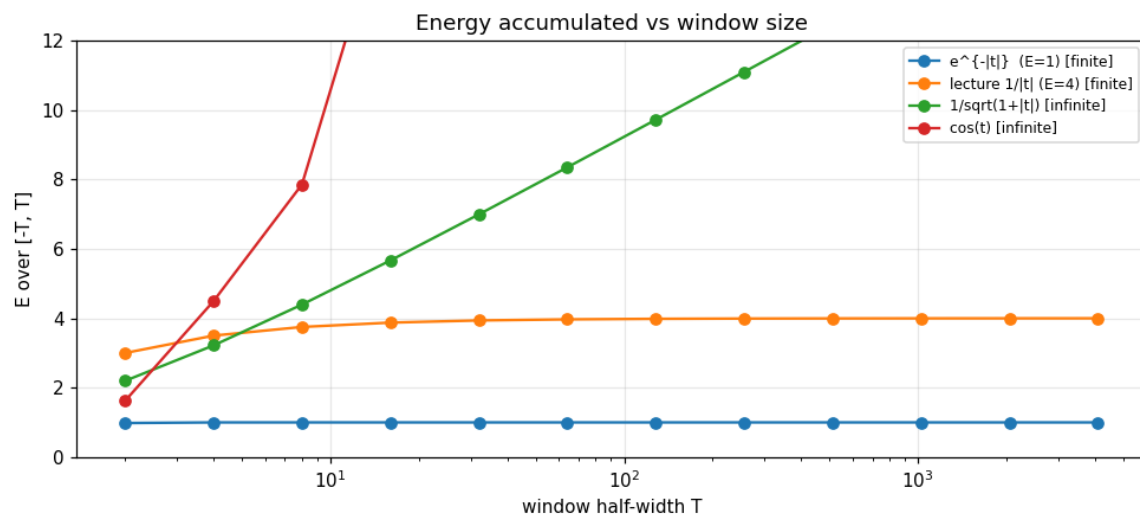
e^{-|t|} (E=1)      -> finite      E(inf) ~ 1
lecture 1/|t| (E=4) -> finite      E(inf) ~ 4

```

```

1/sqrt(1+|t|)      -> infinite  E(inf) ~ inf
cos(t)             -> infinite  E(inf) ~ inf
All assertions passed.

```



Problem 5: An Echo / Multipath Signal and Its Energy (Marks: 10 Suggested time: 30 minutes)

A real receiver hears not just a transmitted pulse $x(t)$ but a train of attenuated, delayed copies (echoes) arriving along different paths:

$$y(t) = \sum_{k=0}^K \alpha^k x(t - kd), \quad 0 < \alpha < 1.$$

This is time shift (delay) plus amplitude scaling, applied repeatedly. The prototype “ping” $x(t)$ is a Gaussian-modulated cosine pulse (provided).

Tasks.

1. Implement `delay(t, x, d)` returning $x(t - d)$ on the same grid by linear interpolation (zero outside).
2. Implement `build_echo(t, x, alpha, d, K)`.
3. Implement `energy_ct` and study the energy of y as the number of echoes K grows, for $\alpha = 0.6$, $d = 1.5$.
4. When the echoes barely overlap, the total energy approaches the geometric limit $E_0/(1 - \alpha^2)$ where E_0 is the single-pulse energy. Compare your measured energy against this prediction and plot both the echoed waveform and the energy-vs- K curve.

Marks: `delay` 2, `build_echo` 3, `energy_ct` 1, convergence study + prediction match + plots 4.

Provided template (fill in the functions marked ANSWER IMPLEMENTATION)

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  T_MIN, T_MAX, N = -2.0, 14.0, 8001
5
6
7  def x_of_t(t: np.ndarray) → np.ndarray:
8      """Prototype 'ping': Gaussian-modulated cosine pulse (given)."""
9      return np.exp(-(t**2) / (2 * 0.25**2)) * np.cos(2 * np.pi * 3.0 * t)
10
11
12  # =====
13  # ANSWER IMPLEMENTATION
14  # =====
15
16  def delay(t, x, d):
17      """Return  $x(t - d)$  on grid  $t$  via linear interpolation (0 outside)."""
18      raise NotImplementedError
19
20
21  def build_echo(t, x, alpha, d, K):
22      """ $y(t) = \sum_{k=0}^K \alpha^k x(t - kd)$ ."""
23      raise NotImplementedError
24
25
26  def energy_ct(t, x):
27      """Integral of  $|x|^2 dt$  (trapezoidal)."""
28      raise NotImplementedError
29
30
31  def plot_signals(t, x, y, K, alpha, d, ax):
32      ax.plot(t, x, color="0.6", lw=1, label="prototype x(t)")
33      ax.plot(t, y, lw=1.6, label=f"echoed y(t), K={K}")
34      ax.set_title(fr"$\alpha$={alpha}, $d$={d}"); ax.grid(alpha=.3)
35      ax.legend(fontsize=8); ax.set_xlabel("t")
36

```

```

37
38 def main():
39     t = np.linspace(T_MIN, T_MAX, N); x = x_of_t(t)
40     alpha, d = 0.6, 1.5
41     Ks = list(range(0, 8)); energies = []
42     for K in Ks:
43         energies.append(energy_ct(t, build_echo(t, x, alpha, d, K)))
44         print(f"K={K}: E(y) = {energies[-1]:.5f}")
45
46     E0 = energy_ct(t, x); predicted = E0 / (1 - alpha**2)
47     print(f"\nE0/(1-alpha^2) = {predicted:.5f}   measured = {energies[-1]:.5f}")
48
49     fig, axes = plt.subplots(2, 1, figsize=(9, 6))
50     plot_signals(t, x, build_echo(t, x, alpha, d, 5), 5, alpha, d, axes[0])
51     axes[1].plot(Ks, energies, "o-", label="measured E(y)")
52     axes[1].axhline(predicted, color="r", ls="--", label="E0/(1-alpha^2)")
53     axes[1].set_xlabel("K"); axes[1].set_ylabel("energy")
54     axes[1].grid(alpha=.3); axes[1].legend(fontsize=8)
55     fig.tight_layout(); plt.show()
56
57
58 if __name__ == "__main__":
59     main()

```

Reference solution

If the echoes are separated enough that cross-terms integrate to nearly zero, the energy of the sum is the sum of the energies: $E(y) \approx E_0 \sum_{k=0}^K \alpha^{2k} \rightarrow E_0/(1 - \alpha^2)$. With $\alpha = 0.6$ that limit is $E_0/(1 - 0.36) = E_0/0.64$. The measured energy climbs monotonically in K and lands within a fraction of a percent of the geometric prediction — a concrete illustration that delaying and attenuating copies of a finite-energy pulse yields another finite-energy signal.

```

1  """
2  Problem 5 - Multipath / echo model.
3  Build  $y(t) = \sum_{k=0}^K \alpha^k * x(t - k*d)$  from a prototype pulse  $x(t)$ 
4  using time-shift (delay), then study its energy as  $K$  grows.
5  Reference solution.
6  """
7  import numpy as np
8  import matplotlib
9  import matplotlib.pyplot as plt
10
11  T_MIN, T_MAX, N = -2.0, 14.0, 8001
12
13
14  def x_of_t(t: np.ndarray) → np.ndarray:
15      """Prototype 'ping': a Gaussian-modulated cosine pulse centred at t=0."""
16      return np.exp(-(t) ** 2) / (2 * 0.25 ** 2)) * np.cos(2 * np.pi * 3.0 * t)
17
18
19  # =====
20  # ANSWER IMPLEMENTATION
21  # =====
22
23  def delay(t: np.ndarray, x: np.ndarray, d: float) → np.ndarray:
24      """Return  $x(t - d)$  sampled on grid  $t$  (a delay to the right for  $d > 0$ ),
25      using linear interpolation, 0 outside the original support."""
26      return np.interp(t - d, t, x, left=0.0, right=0.0)
27
28
29  def build_echo(t: np.ndarray, x: np.ndarray,
30                alpha: float, d: float, K: int) → np.ndarray:
31      """ $y(t) = \sum_{k=0}^K \alpha^k * x(t - k*d)$ ."""
32      y = np.zeros_like(t, dtype=float)
33      for k in range(K + 1):
34          y = y + (alpha ** k) * delay(t, x, k * d)
35      return y
36
37

```

```

38 def energy_ct(t: np.ndarray, x: np.ndarray) → float:
39     """Integral of  $|x|^2 dt$  (trapezoidal)."""
40     return float(np.trapezoid(np.abs(x) ** 2, t))
41
42
43 # -----
44 # Provided plotting
45 # -----
46 def plot_signals(t, x, y, K, alpha, d, ax):
47     ax.plot(t, x, color="0.6", lw=1, label="prototype x(t)")
48     ax.plot(t, y, lw=1.6, label=f"echoed y(t), K={K}")
49     ax.set_title(fr"$\alpha$={alpha},\ d={d}")
50     ax.grid(alpha=.3); ax.legend(fontsize=8); ax.set_xlabel("t")
51
52
53 def main():
54     t = np.linspace(T_MIN, T_MAX, N)
55     x = x_of_t(t)
56
57     alpha, d = 0.6, 1.5
58     Ks = list(range(0, 8))
59     energies = []
60     for K in Ks:
61         y = build_echo(t, x, alpha, d, K)
62         energies.append(energy_ct(t, y))
63
64     for K, E in zip(Ks, energies):
65         print(f"K={K}: E(y) = {E:.5f}")
66
67     # As d is large enough that echoes barely overlap, the total energy tends
68     # to  $E_0 * (1 + \alpha^2 + \alpha^4 + \dots) = E_0 / (1 - \alpha^2)$ .
69     E0 = energy_ct(t, x)
70     predicted = E0 / (1 - alpha ** 2)
71     print(f"\nsingle-pulse energy E0 = {E0:.5f}")
72     print(f"geometric prediction E0/(1-alpha^2) = {predicted:.5f}")
73     print(f"measured at K={Ks[-1]} = {energies[-1]:.5f}")
74
75     fig, axes = plt.subplots(2, 1, figsize=(9, 6))
76     plot_signals(t, x, build_echo(t, x, alpha, d, 5), 5, alpha, d, axes[0])
77     axes[1].plot(Ks, energies, "o-", label="measured E(y)")
78     axes[1].axhline(predicted, color="r", ls="--", label="E0/(1-alpha^2)")
79     axes[1].set_xlabel("number of echoes K"); axes[1].set_ylabel("energy")
80     axes[1].grid(alpha=.3); axes[1].legend(fontsize=8)
81     fig.tight_layout()
82     plt.show()
83
84     # ---- verification ----
85     assert abs(energies[-1] - predicted) < 0.02 * predicted
86     # energy is monotonically increasing in K
87     assert all(energies[i] <= energies[i + 1] + 1e-9 for i in range(len(energies) - 1))
88     print("\nAll assertions passed.")
89
90
91 if __name__ == "__main__":
92     main()

```

Expected console output:

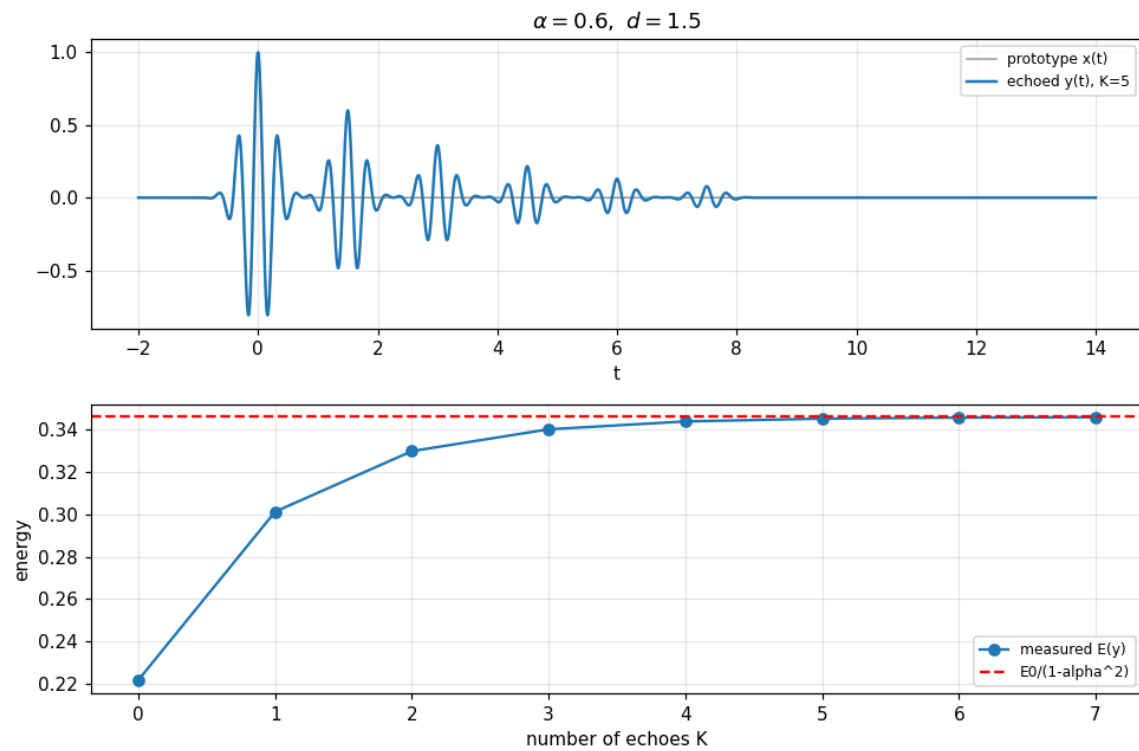
```

K=0: E(y) = 0.22156
K=1: E(y) = 0.30128
K=2: E(y) = 0.32999
K=3: E(y) = 0.34032
K=4: E(y) = 0.34404
K=5: E(y) = 0.34538
K=6: E(y) = 0.34586
K=7: E(y) = 0.34603

single-pulse energy E0 = 0.22156
geometric prediction E0/(1-alpha^2) = 0.34618
measured at K=7 = 0.34603

All assertions passed.

```



Problem 6: The Time-Scaling Energy Law
(minutes)

(Marks: 10 Suggested time: 28

Compressing a signal in time squeezes its energy. If $y(t) = x(at)$ then a change of variables gives

$$E_y = \int |x(at)|^2 dt = \frac{1}{|a|} \int |x(\tau)|^2 d\tau = \frac{E_x}{|a|}.$$

So compression by $a > 1$ divides the energy by a , while stretching by $0 < a < 1$ multiplies it. You will verify this law numerically.

Tasks.

1. Implement `scale_signal(t, x, a)` producing $x(at)$ on the same grid by linear interpolation ($a > 1$ compresses, $0 < a < 1$ stretches, $a < 0$ also reverses).
2. Implement `energy_ct`.
3. Implement `scaling_law_check(t, x, a)` returning $(E_{\text{base}}, E_{\text{scaled}}, |a| E_{\text{scaled}})$; the third number must equal the first for every a .
4. Tabulate the check for $a \in \{0.5, 1, 2, 3, 4, -2, -0.5\}$ on two base signals (a two-sided exponential $e^{-|t|}$ and a triangular pulse). Plot the exponential under several compressions.

Marks: `scale_signal` 3, `energy_ct` 1, law check + table 4, plot 2.

Provided template (fill in the functions marked ANSWER IMPLEMENTATION)

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  T_MIN, T_MAX, N = -12.0, 12.0, 24001
5
6
7  def x_of_t(t): return np.exp(-np.abs(t))          # base signal (given)
8  def x_tri(t):  # second base signal (given)
9      y = np.zeros_like(t, dtype=float); m = np.abs(t) <= 2
10     y[m] = 1 - np.abs(t[m]) / 2; return y
11
12
13 # =====
14 # ANSWER IMPLEMENTATION
15 # =====
16
17 def scale_signal(t, x, a):
18     """Return y(t)=x(a*t) on grid t (a>1 compresses, 0<a<1 stretches, a<0
19     also reverses). Use linear interpolation."""
20     raise NotImplementedError
21
22
23 def energy_ct(t, x):
24     """Integral of |x|^2 dt (trapezoidal)."""
25     raise NotImplementedError
26
27
28 def scaling_law_check(t, x, a):
29     """Return (E_base, E_scaled, E_scaled*|a|). The third value should equal
30     E_base for every a -- that is the law you must confirm."""
31     raise NotImplementedError
32
33
34 def main():
35     t = np.linspace(T_MIN, T_MAX, N)
36     for label, xfunc in [("e^{-|t|}", x_of_t), ("triangle", x_tri)]:
37         x = xfunc(t)
38         print(f"\n=== base: {label} === E_base={energy_ct(t, x):.5f}")
39         print(f"{a':>6}{'E_scaled':>12}{'|a|*E_scaled':>14}")
40         for a in [0.5, 1.0, 2.0, 3.0, 4.0, -2.0, -0.5]:

```

```

41     _, Es, Es_a = scaling_law_check(t, x, a)
42     print(f"{a:6.2f}{Es:12.5f}{Es_a:14.5f}")
43
44     x = x_of_t(t)
45     fig, ax = plt.subplots(figsize=(9, 4))
46     for a in [0.5, 1.0, 2.0, 4.0]:
47         ax.plot(t, scale_signal(t, x, a), label=f"x({a}t)")
48     ax.set_xlim(-6, 6); ax.grid(alpha=.3); ax.legend(fontsize=8)
49     ax.set_xlabel("t"); fig.tight_layout(); plt.show()
50
51
52 if __name__ == "__main__":
53     main()

```

Reference solution

The product $|a| E_{\text{scaled}}$ comes out constant and equal to E_{base} across every scaling factor, including the reversing ones ($a < 0$, where only $|a|$ matters for energy). For $e^{-|t|}$ that constant is $E_{\text{base}} = 1$; for the width-2 triangle it is $4/3$. Watch the grid: stretching ($a < 1$) widens the signal, so the time window must be wide enough to contain the stretched support, otherwise truncation breaks the law.

```

1  """
2  Problem 6 - The time-scaling energy law.
3  If  $y(t) = x(at)$  then  $E_y = E_x / |a|$ . Verify it numerically.
4  Reference solution.
5  """
6  import numpy as np
7  import matplotlib
8  import matplotlib.pyplot as plt
9
10 T_MIN, T_MAX, N = -12.0, 12.0, 24001
11
12
13 def x_of_t(t: np.ndarray) → np.ndarray:
14     """Base energy signal: a two-sided decaying exponential."""
15     return np.exp(-np.abs(t))
16
17
18 def x_tri(t: np.ndarray) → np.ndarray:
19     """A second base signal: triangular pulse of half-width 2."""
20     y = np.zeros_like(t, dtype=float)
21     m = np.abs(t) <= 2
22     y[m] = 1 - np.abs(t[m]) / 2
23     return y
24
25
26 # =====
27 # ANSWER IMPLEMENTATION
28 # =====
29
30 def scale_signal(t: np.ndarray, x: np.ndarray, a: float) → np.ndarray:
31     """Return  $y(t) = x(at)$  sampled on grid  $t$ .  $a > 1$  compresses,  $0 < a < 1$ 
32     stretches,  $a < 0$  also reverses. Uses linear interpolation."""
33     return np.interp(a * t, t, x, left=0.0, right=0.0)
34
35
36 def energy_ct(t: np.ndarray, x: np.ndarray) → float:
37     return float(np.trapezoid(np.abs(x) ** 2, t))
38
39
40 def scaling_law_check(t: np.ndarray, x: np.ndarray, a: float):
41     """Return ( $E_{\text{base}}$ ,  $E_{\text{scaled}}$ ,  $E_{\text{scaled\_times\_}|a|}$ ).
42     The third value should equal  $E_{\text{base}}$  for every  $a$ ."""
43     Eb = energy_ct(t, x)
44     y = scale_signal(t, x, a)
45     Es = energy_ct(t, y)
46     return Eb, Es, Es * abs(a)
47
48

```



```

49 # -----
50 # Provided plotting
51 # -----
52 def main():
53     t = np.linspace(T_MIN, T_MAX, N)
54
55     for label, xfunc in [("e^{-|t|}", x_of_t), ("triangle", x_tri)]:
56         x = xfunc(t)
57         print(f"\n=== base signal: {label} ===")
58         Eb = energy_ct(t, x)
59         print(f"E_base = {Eb:.5f}")
60         print(f"{a':>6} {'E_scaled':>12} {'|a|*E_scaled':>14}")
61         for a in [0.5, 1.0, 2.0, 3.0, 4.0, -2.0, -0.5]:
62             _, Es, Es_a = scaling_law_check(t, x, a)
63             print(f"{a:6.2f} {Es:12.5f} {Es_a:14.5f}")
64
65     # Plot the exponential under a few scalings
66     x = x_of_t(t)
67     fig, ax = plt.subplots(figsize=(9, 4))
68     for a in [0.5, 1.0, 2.0, 4.0]:
69         ax.plot(t, scale_signal(t, x, a), label=f"x({a}t)")
70     ax.set_xlim(-6, 6); ax.grid(alpha=.3); ax.legend(fontsize=8)
71     ax.set_title(r"Compressing $x(t)=e^{-|t|}$ shrinks its energy by $1/|a|$")
72     ax.set_xlabel("t")
73     fig.tight_layout()
74     plt.show()
75
76     # ---- verification ----
77     x = x_of_t(t)
78     Eb = energy_ct(t, x)
79     for a in [0.5, 2.0, 3.0, 4.0, -2.0]:
80         _, _, Es_a = scaling_law_check(t, x, a)
81         assert abs(Es_a - Eb) < 1e-2 * Eb, (a, Es_a, Eb)
82     print("\nAll assertions passed.")
83
84
85 if __name__ == "__main__":
86     main()

```

Expected console output:

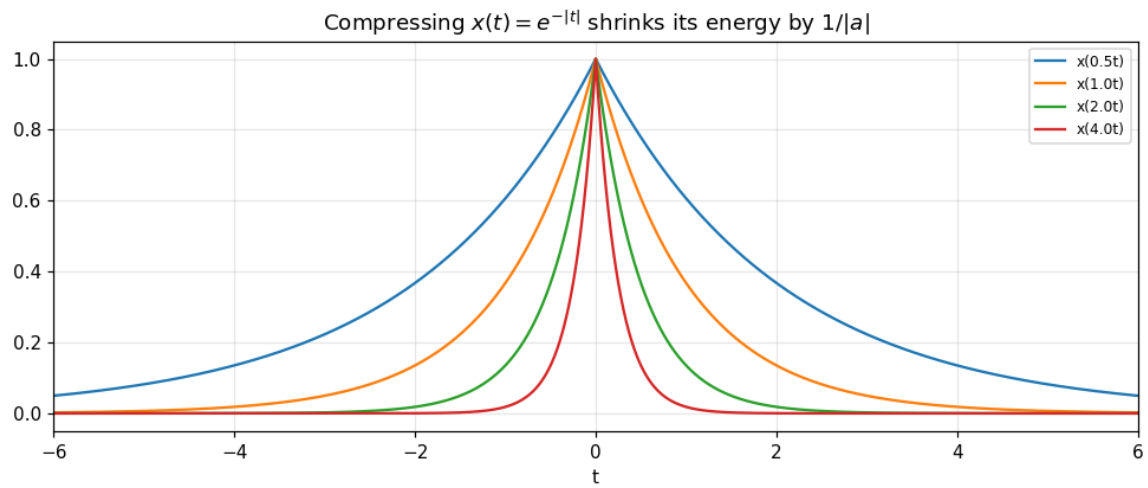
```

=== base signal: e^{-|t|} ===
E_base = 1.00000
  a      E_scaled  |a|*E_scaled
0.50    1.99999    0.99999
1.00    1.00000    1.00000
2.00    0.50000    1.00000
3.00    0.33333    1.00000
4.00    0.25000    1.00001
-2.00    0.50000    1.00000
-0.50    1.99999    0.99999

=== base signal: triangle ===
E_base = 1.33333
  a      E_scaled  |a|*E_scaled
0.50    2.66667    1.33333
1.00    1.33333    1.33333
2.00    0.66667    1.33333
3.00    0.44444    1.33333
4.00    0.33333    1.33334
-2.00    0.66667    1.33333
-0.50    2.66667    1.33333

All assertions passed.

```



Problem 7: Energy Splits Between the Even and Odd Parts

(Marks: 10)

Suggested time: 30 minutes)

Lecture 1 showed every signal decomposes as $x = x_e + x_o$ with $x_e(t) = \frac{1}{2}[x(t) + x(-t)]$ and $x_o(t) = \frac{1}{2}[x(t) - x(-t)]$. A beautiful consequence is that *energy is additive* across this split,

$$E_x = E_{x_e} + E_{x_o},$$

because the cross-energy $\int x_e(t)x_o(t)dt = 0$: the integrand is an even signal times an odd signal, i.e. odd, and an odd signal integrates to zero. You will confirm both facts numerically, in continuous *and* discrete time.

Tasks.

1. Implement `even_odd_ct` and `even_odd_dt`. On a grid symmetric about the origin, $x(-t)$ is simply the reversed sample array.
2. Implement energy and cross-energy for both domains.
3. For the one-sided signals $x(t) = e^{-t}u(t)$ and $x[n] = (0.7)^n u[n]$ (each neither even nor odd), verify $E_x = E_{x_e} + E_{x_o}$ and that the cross-energy is essentially zero.
4. Plot the even and odd parts in both domains.

Marks: even/odd (CT + DT) 3, energies 2, cross-energy 2, verification and plots 3.

Provided template (fill in the functions marked ANSWER IMPLEMENTATION)

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  T_MIN, T_MAX, N = -8.0, 8.0, 16001    # symmetric grid, 0 is a sample
5
6
7  def x_ct(t): return np.where(t >= 0, np.exp(-t), 0.0)      # given
8  def x_dt(n): return np.where(n >= 0, 0.7 ** np.abs(n), 0.0) # given
9
10
11 # =====
12 # ANSWER IMPLEMENTATION
13 # =====
14
15 def even_odd_ct(t, x):
16     """Even and odd parts on a grid symmetric about 0. Hint: x(-t) is the
17     reversed sample array x[::-1]."""
18     raise NotImplementedError
19
20
21 def even_odd_dt(n, x):
22     """Even and odd parts of a DT signal on a symmetric index range."""
23     raise NotImplementedError
24
25
26 def energy_ct(t, x):          raise NotImplementedError
27 def cross_energy_ct(t, a, b): raise NotImplementedError
28 def energy_dt(x):             raise NotImplementedError
29 def cross_energy_dt(a, b):    raise NotImplementedError
30
31
32 def main():
33     t = np.linspace(T_MIN, T_MAX, N); x = x_ct(t)
34     xe, xo = even_odd_ct(t, x)
35     print("=== Continuous-time ===")
36     print(f"E_x          = {energy_ct(t, x):.6f}")
37     print(f"E_xe+E_xo     = {energy_ct(t, xe)+energy_ct(t, xo):.6f}")
38     print(f"cross-eng    = {cross_energy_ct(t, xe, xo):.2e}")
39

```

```

40 n = np.arange(-40, 41); xd = x_dt(n)
41 xde, xdo = even_odd_dt(n, xd)
42 print("\n=== Discrete-time ===")
43 print(f"E_x      = {energy_dt(xd):.6f}")
44 print(f"E_xe+E_xo  = {energy_dt(xde)+energy_dt(xdo):.6f}")
45 print(f"cross-eng  = {cross_energy_dt(xde, xdo):.2e}")
46
47 # TODO: plot the CT and DT even/odd parts.
48 plt.show()
49
50
51 if __name__ == "__main__":
52     main()

```

Reference solution

For $x(t) = e^{-t}u(t)$ the total energy is $\int_0^\infty e^{-2t}dt = \frac{1}{2}$, and it splits almost evenly between the even and odd parts. The measured cross-energy is $\sim 10^{-16}$ (round-off), confirming orthogonality. The discrete case is even cleaner: with the sample at $n = 0$ shared symmetrically, $E_{x_e} + E_{x_o}$ matches E_x to full machine precision and the cross-energy vanishes exactly. This orthogonality is the discrete/continuous shadow of the Pythagorean theorem for signals.

```

1  """
2  Problem 7 - Energy is shared between the even and odd parts.
3  Show numerically that  $E_x = E_{x_e} + E_{x_o}$  and that the cross-energy
4  integral( $x_e * x_o$ ) = 0 (the even and odd parts are orthogonal), for both a
5  continuous-time and a discrete-time signal.
6  Reference solution.
7  """
8  import numpy as np
9  import matplotlib
10 import matplotlib.pyplot as plt
11
12 T_MIN, T_MAX, N = -8.0, 8.0, 16001 # symmetric grid, 0 is a sample
13
14
15 def x_ct(t: np.ndarray) → np.ndarray:
16     """One-sided decaying exponential  $x(t)=e^{-t}$  for  $t \geq 0$ , else 0.
17     This is neither even nor odd, so both parts are nontrivial."""
18     return np.where(t >= 0, np.exp(-t), 0.0)
19
20
21 def x_dt(n: np.ndarray) → np.ndarray:
22     """One-sided geometric  $x[n] = (0.7)^n$  for  $n \geq 0$ , else 0."""
23     return np.where(n >= 0, 0.7 ** np.abs(n), 0.0)
24
25
26 # =====
27 # ANSWER IMPLEMENTATION
28 # =====
29
30 def even_odd_ct(t: np.ndarray, x: np.ndarray):
31     """Even and odd parts of a CT signal sampled on a grid symmetric about 0.
32      $x_e(t)=(x(t)+x(-t))/2$ ,  $x_o(t)=(x(t)-x(-t))/2$ .  $x(-t)$  obtained by flipping
33     the sample array (valid on a symmetric grid)."""
34     xr = x[::-1] # samples of  $x(-t)$ 
35     xe = 0.5 * (x + xr)
36     xo = 0.5 * (x - xr)
37     return xe, xo
38
39
40 def even_odd_dt(n: np.ndarray, x: np.ndarray):
41     """Even and odd parts of a DT signal on a symmetric index range."""
42     xr = x[::-1] #  $x[-n]$ 
43     xe = 0.5 * (x + xr)
44     xo = 0.5 * (x - xr)
45     return xe, xo
46
47

```

```

48 def energy_ct(t, x):
49     return float(np.trapezoid(np.abs(x) ** 2, t))
50
51
52 def cross_energy_ct(t, a, b):
53     return float(np.trapezoid(a * b, t))
54
55
56 def energy_dt(x):
57     return float(np.sum(np.abs(x) ** 2))
58
59
60 def cross_energy_dt(a, b):
61     return float(np.sum(a * b))
62
63
64 # -----
65 # Provided plotting
66 # -----
67 def main():
68     # ----- continuous -----
69     t = np.linspace(T_MIN, T_MAX, N)
70     x = x_ct(t)
71     xe, xo = even_odd_ct(t, x)
72
73     Ex = energy_ct(t, x)
74     Exe = energy_ct(t, xe)
75     Exo = energy_ct(t, xo)
76     cross = cross_energy_ct(t, xe, xo)
77     print("=== Continuous-time x(t)=e^{-t}u(t) ===")
78     print(f"E_x = {Ex:.6f}")
79     print(f"E_xe + E_xo = {Exe + Exo:.6f} (E_xe={Exe:.6f}, E_xo={Exo:.6f})")
80     print(f"cross-energy = {cross:.2e} (should be ~0)")
81
82     # ----- discrete -----
83     n = np.arange(-40, 41)
84     xd = x_dt(n)
85     xde, xdo = even_odd_dt(n, xd)
86     Exd = energy_dt(xd)
87     Exde = energy_dt(xde)
88     Exdo = energy_dt(xdo)
89     crossd = cross_energy_dt(xde, xdo)
90     print("\n=== Discrete-time x[n]=(0.7)^n u[n] ===")
91     print(f"E_x = {Exd:.6f}")
92     print(f"E_xe + E_xo = {Exde + Exdo:.6f}")
93     print(f"cross-energy = {crossd:.2e} (should be ~0)")
94
95     fig, ax = plt.subplots(1, 2, figsize=(11, 4))
96     ax[0].plot(t, x, "k--", lw=1, label="x(t)")
97     ax[0].plot(t, xe, label="xe(t)")
98     ax[0].plot(t, xo, label="xo(t)")
99     ax[0].set_xlim(-5, 5); ax[0].grid(alpha=.3); ax[0].legend(fontsize=8)
100    ax[0].set_title("CT even/odd parts")
101    m1, s1, b1 = ax[1].stem(n, xde, linefmt="C0-", markerfmt="C0o", label="xe[n]")
102    b1.set_visible(False)
103    m1, s1, b1 = ax[1].stem(n, xdo, linefmt="C1-", markerfmt="C1s", label="xo[n]")
104    b1.set_visible(False)
105    ax[1].set_xlim(-15, 15); ax[1].grid(alpha=.3); ax[1].legend(fontsize=8)
106    ax[1].set_title("DT even/odd parts")
107    fig.tight_layout()
108    plt.show()
109
110    # ---- verification ----
111    assert abs((Exe + Exo) - Ex) < 1e-4 * Ex
112    assert abs(cross) < 1e-6
113    assert abs((Exde + Exdo) - Exd) < 1e-9
114    assert abs(crossd) < 1e-9
115    print("\nAll assertions passed.")
116
117
118 if __name__ == "__main__":
119     main()

```

Expected console output:

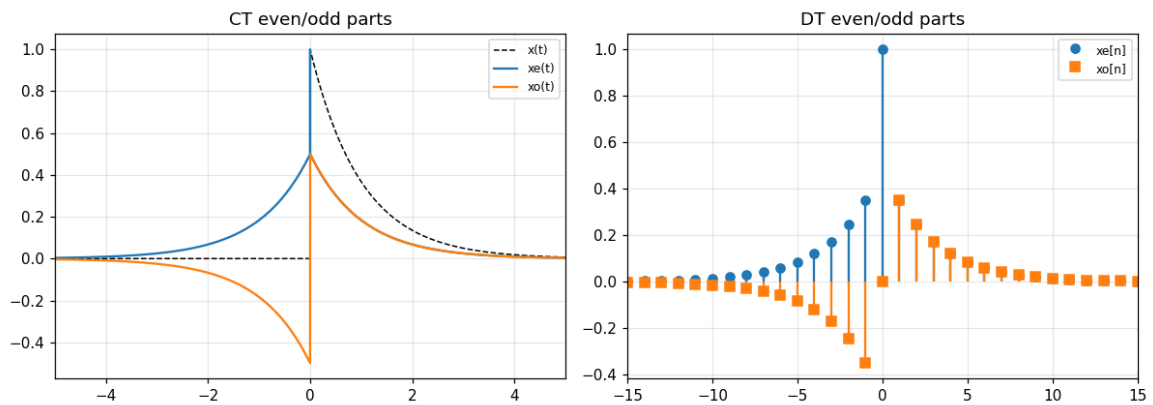
```

=== Continuous-time  $x(t)=e^{-t}u(t)$  ===
E_x      = 0.500500
E_xe + E_xo = 0.500500 (E_xe=0.250750, E_xo=0.249750)
cross-energy = 9.71e-17 (should be ~0)

=== Discrete-time  $x[n]=(0.7)^n u[n]$  ===
E_x      = 1.960784
E_xe + E_xo = 1.960784
cross-energy = 4.93e-18 (should be ~0)

All assertions passed.

```



Problem 8: Average Power of Periodic Signals — One Period Is Enough

(Marks: 10 Suggested time: 27 minutes)

For a periodic signal the lecture claimed that “averaging over one period is enough”: the average power computed over a single fundamental period equals the average power over any whole number of periods. You will test this on four classic periodic waveforms and compare against their exact analytic power.

Tasks.

1. Implement `power_over_interval(xfunc, t0, t1)`: $\frac{1}{t_1 - t_0} \int_{t_0}^{t_1} |x|^2 dt$.
2. Implement `power_one_period` and `power_k_periods`.
3. For $|\sin t|$ (period π , exact $\frac{1}{2}$), a square wave (period 2, exact 1), the lecture half-sine train (period 2π , exact $\frac{1}{4}$), and $\cos^2 t$ (period π , exact $\frac{3}{8}$), show that the one-period power equals the eight-period power and matches the exact value.
4. Plot the half-sine train with one period highlighted.

Marks: `power_over_interval` 3, one/ K -period functions 2, correct match to the four exact values 3, plot 2.

Provided template (fill in the functions marked ANSWER IMPLEMENTATION)

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # =====
5  # ANSWER IMPLEMENTATION
6  # =====
7
8  def power_over_interval(xfunc, t0, t1, n=200001):
9      """Average power of x over [t0,t1]: (1/(t1-t0)) * integral |x|^2 dt."""
10     raise NotImplementedError
11
12
13  def power_one_period(xfunc, T):
14      """Average power over exactly one fundamental period [0, T]."""
15     raise NotImplementedError
16
17
18  def power_k_periods(xfunc, T, K):
19      """Average power over K periods [0, K*T]."""
20     raise NotImplementedError
21
22
23  # -----
24  # Periodic test signals (given)
25  # -----
26  def rectified_sine(t): return np.abs(np.sin(t))
27  def cos_squared(t):    return np.cos(t) ** 2
28  def square_wave(t):    return np.sign(np.sin(np.pi*t)) + (np.sin(np.pi*t) == 0)
29  def half_sine_train(t):
30      tt = np.mod(t, 2*np.pi); return np.where(tt <= np.pi, np.sin(tt), 0.0)
31
32
33  def main():
34      signals = [("|sin t|", rectified_sine, np.pi, 0.5),
35                ("square wave", square_wave, 2.0, 1.0),
36                ("half-sine train", half_sine_train, 2*np.pi, 0.25),
37                ("cos^2 t", cos_squared, np.pi, 3/8)]
38      print(f"{'signal':18s}{'P(1 period)':>13s}{'P(8 periods)':>14s}{'exact':>9s}")
39      for name, f, T, exact in signals:
40          p1 = power_one_period(f, T); pK = power_k_periods(f, T, 8)
41          print(f"{'name':18s}{'p1':13.6f}{'pK':14.6f}{'exact':9.4f}")
42
43      t = np.linspace(0, 4*np.pi, 2000)

```

```

44 fig, ax = plt.subplots(figsize=(9, 3.2))
45 ax.plot(t, half_sine_train(t), lw=1.8)
46 ax.axvspan(0, 2*np.pi, color="orange", alpha=.15, label="one period")
47 ax.grid(alpha=.3); ax.legend(fontsize=8); ax.set_xlabel("t")
48 fig.tight_layout(); plt.show()
49
50
51 if __name__ == "__main__":
52     main()

```

Reference solution

The half-sine train is the “resting” lobe from the slides: a sine lobe on $[0, \pi]$ then silence on $[\pi, 2\pi]$. Its one-period power is $\frac{1}{2\pi} \int_0^\pi \sin^2 t \, dt = \frac{1}{2\pi} \cdot \frac{\pi}{2} = \frac{1}{4}$, matching the lecture. For $\cos^2 t$, the power is $\langle \cos^4 t \rangle = \frac{3}{8}$. In every case the one-period and eight-period numbers agree to six decimals — provided the interval spans a *whole* number of periods; a fractional period would leave a boundary error.

```

1  """
2  Problem 8 - "Averaging over one period is enough."
3  For a periodic signal, verify that the average power computed over a single
4  fundamental period equals the average power over K periods, and compare with
5  the exact analytic values.
6  Reference solution.
7  """
8  import numpy as np
9  import matplotlib
10 import matplotlib.pyplot as plt
11
12
13 # =====
14 # ANSWER IMPLEMENTATION
15 # =====
16
17 def power_over_interval(xfunc, t0: float, t1: float, n: int = 200001) → float:
18     """Average power of x over [t0, t1]:
19         (1/(t1-t0)) * integral_{t0}^{t1} |x(t)|^2 dt    (trapezoidal)."""
20     t = np.linspace(t0, t1, n)
21     x = xfunc(t)
22     energy = np.trapezoid(np.abs(x) ** 2, t)
23     return float(energy / (t1 - t0))
24
25
26 def power_one_period(xfunc, T: float) → float:
27     """Average power over exactly one fundamental period [0, T]."""
28     return power_over_interval(xfunc, 0.0, T)
29
30
31 def power_k_periods(xfunc, T: float, K: int) → float:
32     """Average power over K fundamental periods [0, K*T]."""
33     return power_over_interval(xfunc, 0.0, K * T)
34
35
36 # -----
37 # Periodic test signals (provided)
38 # -----
39 def rectified_sine(t):          # |sin t|, period pi, exact power 1/2
40     return np.abs(np.sin(t))
41
42
43 def square_wave(t):            # +/-1, period 2, exact power 1
44     return np.sign(np.sin(np.pi * t)) + (np.sin(np.pi * t) == 0)
45
46
47 def half_sine_train(t):        # lecture signal: sin on [0,pi], rest on [pi,2pi]
48     tt = np.mod(t, 2 * np.pi)
49     return np.where(tt <= np.pi, np.sin(tt), 0.0)
50
51

```



```

52 def cos_squared(t):          # cos^2 t, period pi, exact power <cos^4> = 3/8
53     return np.cos(t) ** 2
54
55
56 def main():
57     signals = [
58         ("|sin t|",          rectified_sine, np.pi,    0.5),
59         ("square wave",      square_wave,    2.0,        1.0),
60         ("half-sine train",  half_sine_train, 2 * np.pi, 0.25),
61         ("cos^2 t",          cos_squared,     np.pi,      3 / 8),
62     ]
63
64     print(f'{"signal":18s}{\'P(1 period)\':>13s}{\'P(8 periods)\':>14s}{\'exact\':>9s}')
65     for name, f, T, exact in signals:
66         p1 = power_one_period(f, T)
67         pK = power_k_periods(f, T, 8)
68         print(f'{"name":18s}{p1:13.6f}{pK:14.6f}{exact:9.4f}')
69
70     # Plot the half-sine train and highlight one period
71     t = np.linspace(0, 4 * np.pi, 2000)
72     fig, ax = plt.subplots(figsize=(9, 3.2))
73     ax.plot(t, half_sine_train(t), lw=1.8)
74     ax.axvspan(0, 2 * np.pi, color="orange", alpha=.15, label="one period")
75     ax.set_title("Half-sine train: power over one period = power over many")
76     ax.set_xlabel("t"); ax.grid(alpha=.3); ax.legend(fontsize=8)
77     fig.tight_layout()
78     plt.show()
79
80     # ---- verification ----
81     for name, f, T, exact in signals:
82         p1 = power_one_period(f, T)
83         pK = power_k_periods(f, T, 8)
84         assert abs(p1 - pK) < 1e-3, (name, p1, pK)
85         assert abs(p1 - exact) < 2e-3, (name, p1, exact)
86     print("\nAll assertions passed.")
87
88
89 if __name__ == "__main__":
90     main()

```

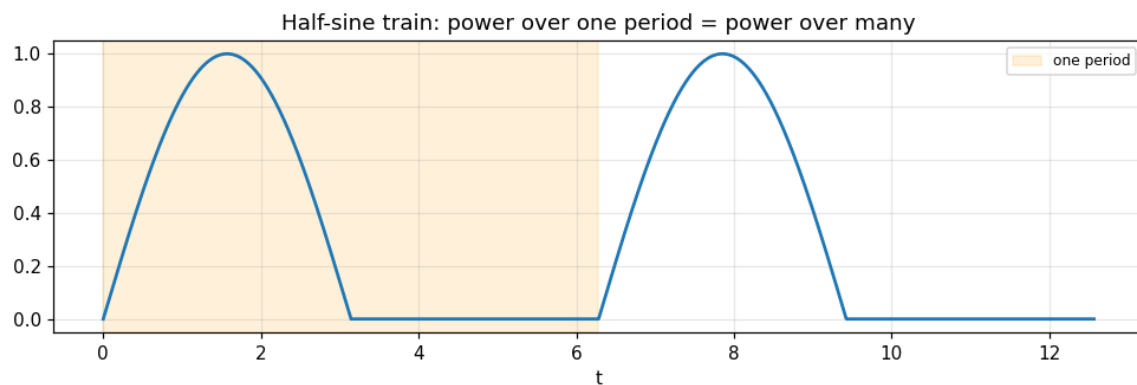
Expected console output:

```

signal          P(1 period)  P(8 periods)  exact
|sin t|         0.500000    0.500000    0.5000
square wave     1.000000    1.000000    1.0000
half-sine train 0.250000    0.250000    0.2500
cos^2 t         0.375000    0.375000    0.3750

All assertions passed.

```



Problem 9: Bridging Discrete and Continuous Energy (Marks: 10 Suggested time: 30 minutes)

The continuous energy $\int |x(t)|^2 dt$ is the limit of the discrete Riemann sum $\Delta t \sum_n |x[n]|^2$ formed from samples taken Δt apart. This is the bridge between the two energy definitions in Lecture 1. You will sample a continuous signal at ever-finer spacings and watch the discrete estimate converge to the true integral.

Tasks.

1. Implement `ct_energy_reference` (a very fine trapezoidal integral used as ground truth).
2. Implement `dt_energy_estimate(xfunc, t0, t1, dt)` returning $\Delta t \sum |x[n]|^2$ over samples spaced Δt apart.
3. Implement `convergence_study` returning the reference, the estimates, and the absolute errors for a list of Δt values.
4. Tabulate the error, estimate the empirical convergence order from the slope of $\log(\text{error})$ vs. $\log \Delta t$, and plot the sampled signal and the error curve. Explain in a comment why the one-sided signal $e^{-0.7t} \cos(5t)u(t)$ (a jump at $t = 0$) gives roughly *first-order* convergence.

Marks: reference 2, estimate 3, study + order estimate 3, plots + explanation 2.

Provided template (fill in the functions marked ANSWER IMPLEMENTATION)

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  T_MIN, T_MAX = -1.0, 8.0
5
6
7  def x_of_t(t):
8      """One-sided decaying cosine  $e^{-0.7t}\cos(5t)u(t)$  (given)."""
9      return np.where(t >= 0, np.exp(-0.7 * t) * np.cos(5.0 * t), 0.0)
10
11
12  # =====
13  # ANSWER IMPLEMENTATION
14  # =====
15
16  def ct_energy_reference(xfunc, t0, t1, n=2_000_001):
17      """High-resolution reference for integral  $|x|^2 dt$  (trapezoidal)."""
18      raise NotImplementedError
19
20
21  def dt_energy_estimate(xfunc, t0, t1, dt):
22      """Sample  $x$  at spacing  $dt$  on  $[t0, t1]$ ; return  $dt * \sum |x[n]|^2$ ."""
23      raise NotImplementedError
24
25
26  def convergence_study(xfunc, t0, t1, dts):
27      """Return (reference, estimates_array, abs_errors_array)."""
28      raise NotImplementedError
29
30
31  def main():
32      dts = np.array([0.5, 0.25, 0.1, 0.05, 0.025, 0.01, 0.005])
33      ref, est, err = convergence_study(x_of_t, T_MIN, T_MAX, dts)
34      print(f"Reference energy = {ref:.8f}\n")
35      print(f"{'dt':>8}{'estimate':>14}{'abs error':>14}")
36      for d, e, er in zip(dts, est, err):
37          print(f"{d:8.3f}{e:14.8f}{er:14.2e}")
38
39      mask = dts <= 0.1
40      slope = np.polyfit(np.log(dts[mask]), np.log(err[mask] + 1e-18), 1)[0]
41      print(f"\nEmpirical convergence order (slope) ~ {slope:.2f}")
42

```

```

43 fig, ax = plt.subplots(figsize=(6, 4))
44 ax.loglog(dts, err, "o-"); ax.grid(alpha=.3, which="both")
45 ax.set_xlabel("dt"); ax.set_ylabel("|estimate - reference|")
46 fig.tight_layout(); plt.show()
47
48
49 if __name__ == "__main__":
50     main()

```

Reference solution

The Riemann estimate converges to the continuous energy as $\Delta t \rightarrow 0$, and the log-log error slope is ≈ 1 : first order. The reason is the jump at $t = 0$ (the signal switches on there). Near a discontinuity the rectangle/trapezoid rule makes an $O(\Delta t)$ error regardless of how smooth the rest is, and that term dominates. Instructive contrast: a fully smooth pulse that decays to zero with all its derivatives (e.g. a Gaussian burst) would converge far faster than any power of Δt — so a signal with a corner or jump is exactly what you want to see the Riemann limit at work.

```

1  """
2  Problem 9 - Bridging discrete and continuous energy.
3  The continuous energy integral  $\int |x(t)|^2 dt$  is the limit of the discrete
4  Riemann sum  $\sum dt * \sum |x[n]|^2$ . Sample  $x(t)$  at spacing  $dt$  and watch the
5  discrete energy estimate converge to the continuous value as  $dt \rightarrow 0$ .
6  Reference solution.
7  """
8  import numpy as np
9  import matplotlib
10 import matplotlib.pyplot as plt
11
12 T_MIN, T_MAX = -1.0, 8.0
13
14 def x_of_t(t: np.ndarray) → np.ndarray:
15     """Energy signal: a one-sided decaying cosine  $e^{-0.7t}\cos(5t) u(t)$ .
16     The jump at  $t=0$  keeps the Riemann convergence at a clean polynomial rate
17     (a fully smooth pulse would converge deceptively fast)."""
18     return np.where(t >= 0, np.exp(-0.7 * t) * np.cos(5.0 * t), 0.0)
19
20
21 # =====
22 # ANSWER IMPLEMENTATION
23 # =====
24
25 def ct_energy_reference(xfunc, t0: float, t1: float, n: int = 2_000_001) → float:
26     """High-resolution reference for integral  $\int |x(t)|^2 dt$  (trapezoidal)."""
27     t = np.linspace(t0, t1, n)
28     return float(np.trapezoid(np.abs(xfunc(t)) ** 2, t))
29
30
31 def dt_energy_estimate(xfunc, t0: float, t1: float, dt: float) → float:
32     """Sample  $x$  at spacing  $dt$  on  $[t0, t1]$  and return the Riemann estimate
33      $dt * \sum |x[n]|^2$  of the continuous energy."""
34     n = np.arange(t0, t1 + dt / 2, dt)
35     x = xfunc(n)
36     return float(dt * np.sum(np.abs(x) ** 2))
37
38
39 def convergence_study(xfunc, t0, t1, dts):
40     """Return (estimates, abs_errors) versus the fine reference."""
41     ref = ct_energy_reference(xfunc, t0, t1)
42     est = np.array([dt_energy_estimate(xfunc, t0, t1, dt) for dt in dts])
43     err = np.abs(est - ref)
44     return ref, est, err
45
46
47 # -----
48 # Provided plotting
49 # -----
50
51 def main():

```

```

52 dts = np.array([0.5, 0.25, 0.1, 0.05, 0.025, 0.01, 0.005])
53 ref, est, err = convergence_study(x_of_t, T_MIN, T_MAX, dts)
54
55 print(f"Continuous reference energy = {ref:.8f}\n")
56 print(f"{'dt':>8}{'estimate':>14}{'abs error':>14}")
57 for d, e, er in zip(dts, est, err):
58     print(f"{d:8.3f}{e:14.8f}{er:14.2e}")
59
60 # Empirical convergence order from the last few points (slope on log-log).
61 mask = dts <= 0.1
62 slope = np.polyfit(np.log(dts[mask]), np.log(err[mask] + 1e-18), 1)[0]
63 print(f"\nEmpirical convergence order (slope) ~ {slope:.2f}")
64
65 fig, ax = plt.subplots(1, 2, figsize=(11, 4))
66 tt = np.linspace(T_MIN, T_MAX, 1500)
67 ax[0].plot(tt, x_of_t(tt), lw=1.2)
68 ns = np.arange(T_MIN, T_MAX + 0.1, 0.2)
69 ax[0].plot(ns, x_of_t(ns), "o", ms=4, label="dt=0.2 samples")
70 ax[0].set_title("x(t) and its samples"); ax[0].grid(alpha=.3)
71 ax[0].legend(fontsize=8); ax[0].set_xlabel("t")
72 ax[1].loglog(dts, err, "o-")
73 ax[1].set_xlabel("sampling step dt"); ax[1].set_ylabel("|estimate - reference|")
74 ax[1].set_title(f"Error vs dt (order ~ {slope:.1f})"); ax[1].grid(alpha=.3, which="both")
75 fig.tight_layout()
76 plt.show()
77
78 # ---- verification ----
79 assert err[-1] < err[0] # finer sampling → smaller error
80 assert abs(est[-1] - ref) < 4e-3 # converges to the CT value
81 assert 0.7 < slope < 2.5 # clean polynomial convergence
82 print("\nAll assertions passed.")
83
84
85 if __name__ == "__main__":
86     main()

```

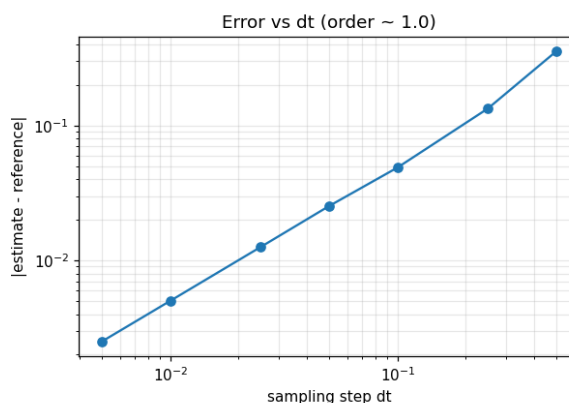
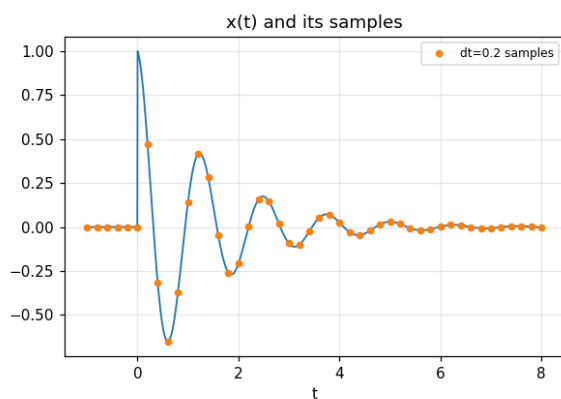
Expected console output:

```
Continuous reference energy = 0.36400150
```

dt	estimate	abs error
0.500	0.71921074	3.55e-01
0.250	0.49775972	1.34e-01
0.100	0.31519971	4.88e-02
0.050	0.38929640	2.53e-02
0.025	0.37657586	1.26e-02
0.010	0.36901446	5.01e-03
0.005	0.36650569	2.50e-03

```
Empirical convergence order (slope) ~ 0.99
```

```
All assertions passed.
```



Problem 10: Building a Waveform from a Single Prototype Pulse (Marks: 10)

Suggested time: 30 minutes)

The transformation toolkit lets you *synthesise* complicated signals from one simple building block. Given an *asymmetric* prototype pulse $p(t)$ (fast rise, slow fall), you must reconstruct a target waveform using only amplitude scaling, time shift, and time reversal of p .

The target is

$$s(t) = \underbrace{p(t)}_{\text{on } [0,4]} + \underbrace{1.5p(8-t)}_{\text{on } [4,8]},$$

i.e. the prototype, plus an amplified *mirror-image* copy placed later in time.

Tasks.

1. Implement the primitives `interp_sample`, `amp_scale`, `shift` ($x(t - t_0)$), and `reverse` ($x(-t)$).
2. Implement `build_target(t, p)` using *only* those primitives. Note that $p(8 - t) = p(-(t - 8))$ must be built by *reversing first, then delaying by 8* — getting the order wrong misplaces the pulse.
3. Verify against the provided analytic target by reporting the MSE (it should be essentially zero), and plot the prototype, the analytic target, and your reconstruction together.
4. In a comment, explain why time *reversal* (not just another shift) is genuinely required here.

Marks: four primitives 4, correct composition in `build_target` 4, verification + plot + explanation 2.

Provided template (fill in the functions marked ANSWER IMPLEMENTATION)

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  T_MIN, T_MAX, N = -4.0, 9.0, 13001
5
6
7  def prototype(t):
8      """Asymmetric continuous pulse: rise slope +1 on [0,1], fall slope -1/3 on
9      [1,4], zero else. Peak 1 at t=1 (given)."""
10     p = np.zeros_like(t, dtype=float)
11     m1 = (t >= 0) & (t <= 1); p[m1] = t[m1]
12     m2 = (t > 1) & (t <= 4); p[m2] = (4 - t[m2]) / 3
13     return p
14
15
16  def target_analytic(t):
17      """Target = p(t) on [0,4] + 1.5*p(8-t) on [4,8] (given, for checking)."""
18      s = np.zeros_like(t, dtype=float)
19      a1 = (t >= 0) & (t <= 1); s[a1] += t[a1]
20      a2 = (t > 1) & (t <= 4); s[a2] += (4 - t[a2]) / 3
21      b1 = (t >= 4) & (t <= 7); s[b1] += 1.5 * (t[b1] - 4) / 3
22      b2 = (t > 7) & (t <= 8); s[b2] += 1.5 * (8 - t[b2])
23      return s
24
25
26  # =====
27  # ANSWER IMPLEMENTATION
28  # =====
29
30  def interp_sample(t, x, tq):
31      """Linear interpolation of sampled x at query times tq, 0 outside."""
32      raise NotImplementedError
33
34
35  def amp_scale(x, c):

```

```

36     """Amplitude scaling c*x."""
37     raise NotImplementedError
38
39
40 def shift(t, x, t0):
41     """Time shift  $x(t - t_0)$  (delay for  $t_0 > 0$ ), on grid  $t$ ."""
42     raise NotImplementedError
43
44
45 def reverse(t, x):
46     """Time reversal  $x(-t)$ , on grid  $t$ ."""
47     raise NotImplementedError
48
49
50 def build_target(t, p):
51     """Reconstruct target using ONLY amp_scale, shift, reverse on p.
52     Copy A =  $p(t)$ ; Copy B =  $1.5 * p(8 - t)$  = reverse first, then delay by 8."""
53     raise NotImplementedError
54
55
56 def main():
57     t = np.linspace(T_MIN, T_MAX, N); p = prototype(t)
58     s_built = build_target(t, p); s_true = target_analytic(t)
59     print(f"MSE(built, analytic) = {np.mean((s_built-s_true)**2):.3e}")
60
61     fig, ax = plt.subplots(figsize=(9, 3.6))
62     ax.plot(t, p, color="0.6", lw=1, label="prototype p(t)")
63     ax.plot(t, s_true, "k--", lw=2, label="target (analytic)")
64     ax.plot(t, s_built, lw=1.6, label="built from p(t)")
65     ax.set_xlim(-1, 9); ax.grid(alpha=.3); ax.legend(fontsize=8); ax.set_xlabel("t")
66     fig.tight_layout(); plt.show()
67
68
69 if __name__ == "__main__":
70     main()

```

Reference solution

Because the prototype is *asymmetric* (rise over width 1, fall over width 3), its time-reversed version has the opposite profile — slow rise, fast fall — which *no* shift of p can produce. That is why copy B needs a genuine reversal. The order matters: $p(8 - t) = p(-(t - 8))$ means reverse to get $p(-t)$, *then* delay by 8; reversing a signal already shifted by 8 would instead give $p(-t - 8)$, landing the pulse in the wrong place. Since the target is continuous and piecewise linear, linear interpolation reproduces it essentially exactly ($\text{MSE} \sim 10^{-34}$).

```

1  """
2  Problem 10 - Building a signal out of one prototype pulse.
3  Using only amplitude scaling, time shift, and time reversal of a single
4  ASYMMETRIC prototype pulse  $p(t)$ , reconstruct a specified target waveform and
5  verify it against the analytic target.
6  Reference solution.
7  """
8  import numpy as np
9  import matplotlib
10 import matplotlib.pyplot as plt
11
12 T_MIN, T_MAX, N = -4.0, 9.0, 13001
13
14
15 def prototype(t: np.ndarray) → np.ndarray:
16     """Asymmetric continuous pulse: fast rise (slope +1 over  $[0,1]$ ) then slow
17     fall (slope -1/3 over  $[1,4]$ ); zero elsewhere. Peak height 1 at  $t=1$ .
18     Because it is NOT symmetric, time reversal produces a genuinely different
19     shape (slow rise, fast fall) that no pure shift can reproduce."""
20     p = np.zeros_like(t, dtype=float)
21     m1 = (t >= 0) & (t <= 1); p[m1] = t[m1]
22     m2 = (t > 1) & (t <= 4); p[m2] = (4 - t[m2]) / 3
23     return p
24

```

```

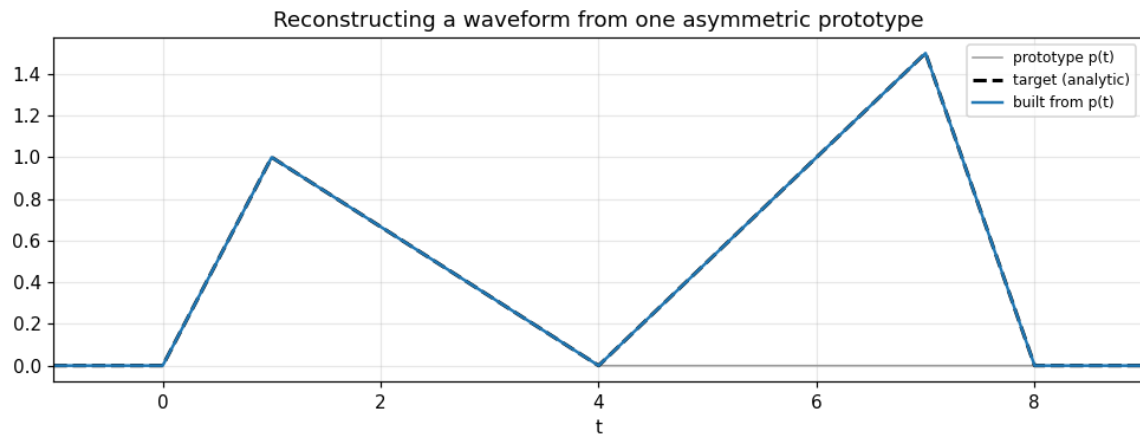
25
26 def target_analytic(t: np.ndarray) → np.ndarray:
27     """Target to reproduce:
28         Copy A = p(t) on [0, 4]
29         Copy B = 1.5 * p(8 - t) on [4, 8] (a reversed, shifted,
30                                         amplified copy)
31     Fully continuous piecewise-linear waveform."""
32     s = np.zeros_like(t, dtype=float)
33     a1 = (t >= 0) & (t <= 1); s[a1] += t[a1]
34     a2 = (t > 1) & (t <= 4); s[a2] += (4 - t[a2]) / 3
35     b1 = (t >= 4) & (t <= 7); s[b1] += 1.5 * (t[b1] - 4) / 3
36     b2 = (t > 7) & (t <= 8); s[b2] += 1.5 * (8 - t[b2])
37     return s
38
39
40 # =====
41 # ANSWER IMPLEMENTATION
42 # =====
43
44 def interp_sample(t, x, tq):
45     return np.interp(tq, t, x, left=0.0, right=0.0)
46
47
48 def amp_scale(x: np.ndarray, c: float) → np.ndarray:
49     """Amplitude scaling: c * x."""
50     return c * x
51
52
53 def shift(t: np.ndarray, x: np.ndarray, t0: float) → np.ndarray:
54     """Time shift (delay for t0>0): x(t - t0)."""
55     return interp_sample(t, x, t - t0)
56
57
58 def reverse(t: np.ndarray, x: np.ndarray) → np.ndarray:
59     """Time reversal: x(-t)."""
60     return interp_sample(t, x, -t)
61
62
63 def build_target(t: np.ndarray, p: np.ndarray) → np.ndarray:
64     """Reconstruct s(t) from the prototype p(t) using only amp_scale, shift,
65     and reverse.
66
67         Copy A : p(t) → p itself
68         Copy B : 1.5 * p(8 - t)
69         p(8 - t) = p(-(t - 8)) = reverse first, THEN delay by 8.
70     """
71     A = p
72     rev = reverse(t, p) # p(-t)
73     B = amp_scale(shift(t, rev, 8.0), 1.5) # 1.5 * p(-(t - 8)) = 1.5 p(8 - t)
74     return A + B
75
76
77 # -----
78 # Provided plotting
79 # -----
80 def main():
81     t = np.linspace(T_MIN, T_MAX, N)
82     p = prototype(t)
83
84     s_built = build_target(t, p)
85     s_true = target_analytic(t)
86     mse = float(np.mean((s_built - s_true) ** 2))
87     print(f"MSE(built, analytic) = {mse:.3e}")
88
89     fig, ax = plt.subplots(figsize=(9, 3.6))
90     ax.plot(t, p, color="0.6", lw=1, label="prototype p(t)")
91     ax.plot(t, s_true, "k--", lw=2, label="target (analytic)")
92     ax.plot(t, s_built, lw=1.6, label="built from p(t)")
93     ax.set_xlim(-1, 9); ax.grid(alpha=.3); ax.legend(fontsize=8); ax.set_xlabel("t")
94     ax.set_title("Reconstructing a waveform from one asymmetric prototype")
95     fig.tight_layout()
96     plt.show()
97
98     # ---- verification ----
99     assert mse < 1e-4, mse
100     print("All assertions passed.")

```

```
101  
102  
103 if __name__ == "__main__":  
104     main()
```

Expected console output:

```
MSE(built, analytic) = 5.270e-34  
All assertions passed.
```



End of problem set — ten problems, ten reference solutions. Every solution here was executed and its assertions verified.